Universidad Tecnológica de La Habana

“José Antonio Echeverría”

Facultad de Ingeniería Informática



Informe de Práctica Profesional II

Implementación de nuevo algoritmo de agrupamiento basado en densidad en la Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos

Autor:

Maykel Kevin Díaz Menéndez

Tutores:

Ing. Eric Ramos Aragón

Ing. Ananda de la Caridad Morales Morales

La Habana, Cuba

Septiembre 2026

Resumen

El Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos (MDVRP) constituye una variante compleja dentro de los problemas de optimización combinatoria. En la práctica, este problema suele resolverse dividiéndolo en dos etapas: asignar los clientes a los depósitos y construir las rutas desde cada depósito. La Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos (BHAVRP) es un componente de software diseñado para resolver esta primera fase de asignación. La biblioteca presenta una arquitectura modular que encapsula algoritmos de agrupamiento clasificados en particionales, jerárquicos y basados en densidad, incluye un sistema de tratamiento de excepciones y un conjunto de métricas de evaluación internas. Actualmente, BHAVRP cuenta con un solo algoritmo de agrupamiento basado en densidad. Se propone la incorporación de un nuevo algoritmo de agrupamiento basado en densidad HDBSCAN. Este algoritmo de agrupamiento es adaptado para resolver la fase de asignación de clientes a depósitos incorporando restricciones de capacidad de los depósitos y mecanismos de asignación controlada usando como base la implemetación de la biblioteca JSAT. Por último, se realiza un estudio experimental que permite garantizar el correcto funcionamiento de los módulos implementados, así como realizar un análisis de correlación entre las metricas internas y las rutas obtenidas por los clústeres. Para ello, se utilizan los criterios definidos por las métricas de evaluación que ya tiene BHAVRP y la Biblioteca de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos (BHCVRP).

Palabras clave: algoritmos de agrupamiento, BHAVRP, HDBSCAN, MDVRP.

Abstract

The Multi-Depot Vehicle Routing Problem (MDVRP) constitutes a complex variant within combinatorial optimization problems. In practice, this problem is often solved by dividing it into two stages: assigning customers to depots and building routes from each depot. The Assignment Heuristics Library for Multi-Depot Vehicle Routing Problems (BHAVRP) is a software component designed to solve this first assignment phase. The library features a modular architecture that encapsulates clustering algorithms classified into partitional, hierarchical, and density-based, includes an exception handling system, and a set of internal evaluation metrics. Currently, BHAVRP has only one density-based clustering algorithm. The incorporation of a new density-based clustering algorithm, HDBSCAN, is proposed. This clustering algorithm is adapted to solve the customer-to-depot assignment phase by incorporating depot capacity constraints and controlled assignment mechanisms, using the implementation from the JSAT library as a basis. Finally, an experimental study is conducted to ensure the correct functioning of the implemented modules, as well as to perform a correlation analysis between the internal metrics and the routes obtained by the clusters. For this purpose, the criteria defined by the evaluation metrics already present in BHAVRP and the Construction Heuristics Library for Vehicle Routing Problems (BHCVRP) are used.

Keywords: BHAVRP, clustering algorithms, HDBSCAN, MDVRP.

Índice general

Introducción **1**

1. Fundamentos teóricos del problema de planificación de rutas y su solución mediante diferentes métodos **6**

1.1. Problemas de Planificación de Rutas de Vehículos 6

1.1.1. Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos 7

1.2. Algoritmos heurísticos 11

1.3. Algoritmos de agrupamiento para la fase de asignación de clientes a depósitos . 15

1.4. Biblioteca de Heurísticas de Asignación en la Planificación de Rutas de Vehículos con Múltiples Depósitos 19

1.4.1. El problema del ruido en los algoritmos de BHAVRP 26

1.5. Conclusiones parciales 28

2. Incorporación de algoritmo a la Biblioteca de Heurísticas de Asignación del Problema de Planificación de Rutas con Múltiples Depósitos **29**

2.1. Análisis y selección del algoritmo basado en densidad 29

2.2. Algoritmo HDBSCAN 33

2.3. Adaptación de HDBSCAN al contexto del MDVRP 36

2.3.1. Mecanismo de inicialización y configuración 36

2.3.2. Construcción de la jerarquía y extracción de clústeres 37

2.3.3. Asignación de clústeres a depósitos 37

2.3.4. Manejo del ruido y reasignación individual 38

2.4. Incorporación del algoritmo propuesto a BHAVRP 40

2.5. Conclusiones parciales 49

3. Análisis experimental **50**

3.1. Pruebas funcionales 50

3.1.1. Pruebas del método *distance* heredado de AbstractDensity 50

3.1.2. Pruebas del método *getNeighbours* de la clase AbstractDensity 51

3.1.3. Pruebas del método *findNearestDepotIndice* de la clase HDBSCAN . . 53

3.2. Pruebas unitarias de caja negra 54

3.3. Comparación entre los algoritmos de agrupamiento 56

3.3.1. Análisis estadístico 58

3.4. Análisis de correlación entre métricas internas y calidad de rutas 62

3.5. Conclusiones parciales 70

Referencias Bibliográficas **71**

Índice de figuras

1.1. Representación gráfica de MDVRP. 10

1.2. Técnicas utilizadas para la solución de MDVRP [39]. 12

1.3. Representación gráfica de algoritmos de agrupamiento particionales [50]. . . . 17

1.4. Representación gráfica de algoritmos de agrupamiento jerárquicos [50]. 18

1.5. Representación gráfica de algoritmos de agrupamiento basados en densidad [50]. 18

1.6. Diagrama de actividades del flujo de BHAVRP. 24

1.7. Diagrama de arquitectura n-capas con enfoque en reutilización de BHAVRP. . . 25

1.8. Asignación generada por el algoritmo TCC sobre una instancia real. 27

1.9. Asignación generada por el algoritmo FF sobre una instancia real. 27

2.1. Diagrama de flujo del algoritmo HDBSCAN. 35

2.2. Diagrama de flujo del paso final del algoritmo HDBSCAN adaptado. 39

2.3. Diagrama de secuencia del algoritmo HDBSCAN. 43

2.4. Diagrama de arquitectura actualizado. 44

2.5. Diagrama de clases del paquete *Clustering*. 46

3.1. Resultados de las pruebas del método distance para HDBSCAN. 51

3.2. Resultados de las pruebas del método getNeighbours. 52

3.3. Resultados de las pruebas del método findNearestDepotIndice. 54

Índice de tablas

1.1. Comparación entre las herramientas que resuelven el MDVRP. 22

2.1. Tabla comparativa de algoritmos de agrupamiento basados en densidad. 31

2.2. Comparación entre DBSCAN y HDBSCAN. 32

2.3. Clases implementadas para la incorporación de los nuevos algoritmos de agrupamiento. 41

3.1. Pruebas unitarias de caja negra del algoritmo HDBSCAN. 55

3.2. Descripción de las instancias. 57

3.3. Descripción de las nuevas instancias. 58

3.4. Resultados de la prueba de Friedman NxN. 60

3.5. Resumen de las métricas incorporadas en BHAVRP. 64

3.6. Correlación entre métricas internas y costo total (heurística SaveParallel). . . . 65

3.7. Correlación entre métricas internas y costo total (heurística SaveSequential). . . 65

3.8. Correlación entre métricas internas y costo total (heurística Sweep). 66

3.9. Correlación entre métricas internas y costo total (heurística CMT). 66

3.10. Correlación entre métricas internas y costo total (heurística MoleJameson). . . 67

3.11. Correlación entre métricas internas y costo total (heurística NearestNeighborWithRLC). 67

3.12. Correlación entre métricas internas y costo total (heurística NearestNeighborWithRLC con RLC=1). 68

Introducción

La gestión de la distribución, el transporte y la planificación de rutas son temas de gran importancia que han sido abordados a nivel internacional. Son ramas de la logística donde yacen numerosas oportunidades de mejora y por esta razón ha recibido especial atención motivando un intenso trabajo teórico en el desarrollo de modelos y algoritmos. En este contexto surgen los Problemas de Planificación de Rutas de Vehículos (*Vehicle Routing Problems*, VRP) [1].

En [2] se introduce el *"Truck Dispatching Problem"*, modelando cómo una flota de camiones homogéneos podría satisfacer la demanda de petróleo de varias estaciones de servicio desde un depósito central y con una distancia mínima recorrida. Cinco años después, en [3] se generaliza este a un problema de optimización lineal que se encuentra comúnmente en el ámbito de la logística y el transporte. El VRP es un problema NP-duro[4] por lo que se consideran muy difíciles de resolver. Los problemas de la vida real son considerablemente más grandes y complejos que los escenarios académicos, por lo que las heurísticas y metaheurísticas son necesarias para obtener buenos resultados. Estos algoritmos no garantizan encontrar la solución óptima, pero sí proporcionan tiempos computacionales razonables. La cantidad de métodos de solución introducidos en la literatura académica ha crecido en las últimas décadas [5].

El Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos (*Multi-Depot Vehicle Routing Problems*, MDVRP) es una variante del VRP clásico que incorpora varios depósitos con una ubicación predefinida. Cada depósito cuenta con una flota de vehículos de capacidad homogénea que tiene como objetivo satisfacer la demanda de un conjunto de clientes dispersos geográficamente. Cada vehículo debe comenzar y terminar su ruta en el mismo depósito, y cada cliente debe ser visitado una única vez. Teniendo en cuenta la distancia entre la ubicación de los clientes y depósitos, el MDVRP busca asignar un cliente a cada depósito y se construye una ruta para cada vehículo dentro de un depósito para atender dichos clientes. El objetivo al crear la ruta es reducir al mínimo el costo total del recorrido [6].

Durante un largo período de tiempo, los métodos heurísticos basados en la búsqueda tabú han sido los más empleados en la literatura [6]. Sin embargo, este problema genera una gran cantidad de combinaciones que se refleja en los altos tiempos computacionales para obtener la solución. El potencial de las técnicas de análisis de clústeres para la solución de los MDVRP ha sido reconocido por autores como [2], [7], [8] y [9]. El análisis de clústeres estudia la división de objetos en grupos basado en una o varias características. Un grupo se define como una zona con muchos objetos parecidos muy cercanos entre sí, que está separada de otros grupos por zonas donde casi no hay objetos. Esta definición de grupo es bastante acorde a la agrupación de clientes por lo que se propone usar el análisis de clústeres en la solución del MDVRP, específicamente en la fase de asignación de clientes a depósitos [7]. En [10] se describe la técnica ”divide y vencerás”, utilizada para reducir la complejidad computacional en el empleo heurísticas para resolver el MDVRP.

Sin embargo, aún existen desafíos en cuanto a la eficiencia computacional debido a los altos tiempos y la calidad de las agrupaciones dependiendo del escenario [7]. En la literatura se destacan algoritmos como *Density-Based Spatial Clustering of Applications with Noise* (DBSCAN)[11] y *K-Means*[12] que han sido probados y utilizados en el ámbito del MDVRP, mostrando su utilidad para la fase de asignación de clientes a depósitos [13, 14, 15].

En este contexto, la Universidad Tecnológica de la Habana “José Antonio Echeverría” (CUJAE) ha desarrollado la Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos (BHAVRP). Este componente está diseñado para resolver la fase de asignación en MDVRP utilizando una arquitectura modular y 20 algoritmos repartidos en heurísticas clásicas de asignación y algoritmos de agrupamiento de Minería de Datos adaptados al contexto de los MDVRP [16]. Esta alternativa gratuita y de código abierto supone una ventaja frente a otras herramientas que no tienen una colección de algoritmos tan extensa o son software comercial.

Esta biblioteca, en la categoría de algoritmos de agrupamiento, incluye algoritmos particionales, algoritmos jerárquicos y un algoritmo basado en densidad. En una investigación anterior [17], se presentó los resultados de algoritmos de agrupamiento particionales y heurísticas clásicas de asignación. Sin embargo, estos algoritmos presentaron una tendencia a asignar clientes a depósitos sin considerar adecuadamente la densidad geográfica, lo que generó clústeres con clientes muy alejados del depósito asignado. Esto se conoce en la literatura como ruido o asignaciones atípicas, lo cual puede incrementar los costos de transporte y afecta la calidad de las rutas posteriores. La literatura reconoce que los algoritmos basados en densidad, como DBSCAN, son capaces de identificar y aislar estos puntos atípicos [11], ya que este tipo de algoritmos parten de la idea de que un clúster es una región de alta densidad de puntos separadas de otras regiones densas por zonas de más baja densidad.

Debido a la situación problemática antes descrita, se identifica como problema a resolver que los algoritmos de agrupamiento particionales y las heurísticas clásicas de asignación implementadas en BHAVRP generan clústeres con presencia de ruido, lo que impacta negativamente en la calidad de las rutas posteriores.

En este contexto, el objeto de estudio de este trabajo es la fase de asignación dentro de los MDVRP y la evaluación de la calidad de los algoritmos de agrupamiento. El campo de acción se enfoca en la incorporación y adaptación de algoritmos de agrupamiento basados en densidad en la fase de asignación de clientes a depósitos de BHAVRP.

Para dar solución al problema planteado se traza como objetivo general: Desarrollar una nueva versión de BHAVRP que incluya más algoritmos de agrupamiento basados en densidad. Para dar cumplimiento al objetivo general se definieron los siguientes objetivos específicos con sus respectivas tareas:

1. Identificar algoritmos de agrupamiento utilizados para resolver la fase de asignación de los problemas de planificación de rutas de vehículos con múltiples depósitos.

- Realizar búsqueda bibliográfica de las características de los problemas de planificación de rutas de vehículos con múltiples depósitos y sus aplicaciones prácticas.
- Realizar búsqueda bibliográfica de las propuestas de solución de MDVRP donde se realice la fase de asignación de clientes a depósitos usando como vía de resolución los algoritmos de agrupamiento.

2. Implementar en BHAVRP un nuevo algoritmo de agrupamiento basado en densidad.
    - Analizar algoritmos de agrupamiento basados en densidad en cuanto a su funcionamiento para su incorporación en la nueva versión de la biblioteca.
    - Seleccionar el algoritmo de agrupamiento que se van a incorporar a la biblioteca para resolver la fase de asignación del MDVRP.
    - Implementar y adaptar el algoritmo seleccionado teniendo en cuenta las características que presentan.
3. Validar el funcionamiento del algoritmo mediante la ejecución de experimentos.
    - Realizar pruebas unitarias, donde se muestre que el algoritmo implementado funcione correctamente.
    - Realizar pruebas estadísticas no paramétricas entre los resultados obtenidos por las heurísticas de construcción usando los resultados de los algoritmos de agrupamiento de BHAVRP.
    - Realizar una prueba de Pearson y una prueba de Spearman para validar si existe una correlación positiva entre las métricas internas de BHAVRP y la calidad de las rutas obtenidas por las heurísticas de construcción.

El cumplimiento del objetivo general tiene como valor práctico un aporte significativo en la resolución de la fase de asignación del problema de planificación de rutas de vehículos con múltiples depósitos. Los resultados tienen valor científico y tecnológico ya que se pone a

disposición de la comunidad científica un componente de software que brinda nuevos algoritmos para la solución de MDVRP.

Para alcanzar los objetivos mencionados, el trabajo se encuentra estructurado en tres capítulos, incluyendo secciones importantes como conclusiones, recomendaciones y referencias bibliográficas. El Capítulo 1: Fundamentos teóricos del problema de planificación de rutas y su solución mediante diferentes métodos donde se presenta el estado del arte del MDVRP, las estrategias de solución existentes, así como un análisis comparativo de bibliotecas. Capítulo 2: Incorporación de algoritmo a la Biblioteca de Heurísticas de Asignación del Problema de Planificación de Rutas con Múltiples Depósitos describe los algoritmos de agrupamiento basados en densiad candidatos más comentados en la literatura, el algoritmo propuesto detallando el funcionamiento de este según la literatura y las adaptaciones al MDVRP que hay que realizar. Finalmente, el Capítulo 3: Análisis experimental expone varias pruebas unitarias de las funciones más relevantes del el algoritmo implementado, así como un estudio comparativo entre algoritmos de asignación en las heurísticas de construcción aplicados sobre ciertas instancias y un análisis de correlación entre las métricas de validación internas y la calidad de las rutas obtenidas.

Capítulo 1: Fundamentos teóricos del problema de planificación de rutas y su solución mediante diferentes métodos

En este capítulo se describen los aspectos teóricos de la investigación que se encuentran relacionados con los MDVRP. Entre los aspectos que se abordan están las características y variantes del problema, así como los diferentes métodos de solución encontrados en la literatura. Además, se hace énfasis en los algoritmos de agrupamiento como alternativa para la fase de asignación de clientes a depósitos. Por último, se describe la Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos y sus algoritmos existentes.

1.1 Problemas de Planificación de Rutas de Vehículos

El concepto de Problema de Planificación de Rutas de Vehículos (*Vehicle Routing Problem*, VRP) es una generalización del Problema del Viajero (*Traveling Salesman Problem*, TSP). En concreto, el VRP es un problema de optimización combinatoria cuyo objetivo es atender las demandas de clientes que se encuentran dispersos geográficamente sujetos a un conjunto de restricciones. La diferencia radica en que TSP dispone de un mapa con múltiples puntos que un individuo tiene que visitar para volver a su punto de origen realizando la ruta más corta posible. En cambio, el VRP no se limita a solo una persona, sino que pueden existir más de un individuo, en este caso vehículos, que normalmente deben entregar objetos en diversos puntos de la ciudad a clientes. De este modo, el problema se vuelve más complejo que el TSP [18].

Las aplicaciones informáticas para el diseño de rutas se derivan de modelos VRP que atienden diferentes necesidades e incluyen diversas restricciones. Esto ha dado como resultado el surgimiento de diferentes variantes del clásico VRP, de las cuales se pueden mencionar las siguientes:

- VRP con recolección y entrega (VRPPD): estudia el caso donde una empresa debe recolectar y entregar bienes en cantidades específicas para cada cliente visitado [19].
- VRP con flota heterogénea: los vehículos tienen diferentes capacidades de carga [20].
- *Open VRP* (OVRP): se refiere al caso de cuando el vehículo no regresa al depósito de origen, es decir, que se dirige a otro depósito o sitio diferente para cargar productos, en un contexto de flota rentada [21].
- VRP con ventanas de tiempo (VRPTW): plantea que cada cliente tiene que ser atendido de manera obligada dentro de un cierto horario específico [22].
- VRP con depósitos múltiples (MDVRP): implica que la empresa posee diversos depósitos desde los cuales puede abastecer a los clientes [23].
- *Stochastic VRP* (SVRP): Propone que las variables involucradas son desconocidas o aleatorias, e incluso que se pueden encontrar en un rango de probabilidad, tales como el número de clientes, sus demandas, tiempo de servicio, tiempo de viaje, entre otras [24].
- *Periodic VRP* (PVRP): analiza un problema donde los pedidos pueden ser llevados sólo en ciertos días [25].
- Capacitated VRP: estudia el caso donde los vehículos tienen una capacidad máxima de carga y se deben diseñar rutas que satisfagan la demanda de los clientes sin exceder dicha capacidad [26].

1.1.1 Problema de Planificación de Rutas de Vehículos con Múltiples Depósitos

El MDVRP es un problema de optimización combinatoria, categorizado por su complejidad como NP-Duro [27]. Por lo que es difícil encontrar su solución óptima usando métodos exactos en un tiempo polinomial. Esta variante cuenta con varios depósitos y cada uno tiene su propia flota de vehículos [28].

El problema puede ser representado mediante modelos de optimización y se basa en [23], [29], [30], [31], [32], [33]. Sea $G = (V, A)$ un grafo completo, donde V es el conjunto de nodos y A es el conjunto de arcos. Los nodos se dividen en dos subconjuntos: los clientes a ser atendidos, $V_C = \{1, \ldots, N\}$, y los múltiples depósitos $V_D = \{N + 1, \ldots, N + M\}$, con $V_C \cup V_D = V$ y $V_C \cap V_D = \emptyset$. Hay un costo no negativo c_{ij} asociado con cada arco $(i, j) \in A$. La demanda de cada cliente es d_i donde no hay demanda en los nodos de depósito. También hay una flota de K vehículos, cada uno con capacidad Q. El tiempo de servicio en cada cliente i es t_i mientras que el tiempo máximo de duración de la ruta se establece en T. Es posible que se necesite un factor de conversión w_{ij} para transformar el costo c_{ij} en unidades de tiempo. Sin embargo, el costo es el mismo que las unidades de tiempo y distancia, por lo que $w_{ij} = 1$ [34].

A continuación se presenta la formulación matemática [34]:

Las variables binarias x_{ijk} son iguales a 1 cuando el vehículo k visita el nodo j inmediatamente después del nodo i. Las variables auxiliares y_i también se utilizan en las restricciones de eliminación de subtours.

Función Objetivo: Minimizar el costo total.

$$\text{Minimize} \sum_{i=1}^{N+M} \sum_{j=1}^{N+M} \sum_{k=1}^{K} c_{ij} x_{ijk}, \tag{1.1}$$

Restricciones:

Garantiza que cada cliente sea visitado exactamente una vez (flujo entrante).

$$\sum_{i=1}^{N+M} \sum_{k=1}^{K} x_{ijk} = 1 \quad (j = 1, \ldots, N); \tag{1.2}$$

Garantiza que cada cliente sea visitado exactamente una vez (flujo saliente).

$$\sum_{j=1}^{N+M} \sum_{k=1}^{K} x_{ijk} = 1 \quad (i = 1, \ldots, N); \tag{1.3}$$

Conservación de flujo. Lo que entra a un nodo es igual a lo que sale de él para cada vehículo.

$$\sum_{i=1}^{N+M} x_{ihk} - \sum_{j=1}^{N+M} x_{hjk} = 0 \quad (k = 1, \ldots, K; h = 1, \ldots, N + M); \tag{1.4}$$

Restricción de capacidad. La demanda total de la ruta no debe exceder la capacidad del vehículo Q.

$$\sum_{i=1}^{N+M} \sum_{j=1}^{N+M} d_i x_{ijk} \leq Q \quad (k = 1, \ldots, K); \tag{1.5}$$

Restricción de duración de la ruta. El costo total más el tiempo de servicio no debe exceder el tiempo máximo T.

$$\sum_{i=1}^{N+M} \sum_{j=1}^{N+M} (c_{ij} w_{ij} + t_i) x_{ijk} \leq T \quad (k = 1, \ldots, K); \tag{1.6}$$

Verifica la disponibilidad de vehículos (salida del depósito).

$$\sum_{i=N+1}^{N+M} \sum_{j=1}^{N} x_{ijk} \leq 1 \quad (k = 1, \ldots, K); \tag{1.7}$$

Verifica la disponibilidad de vehículos (entrada al depósito).

$$\sum_{j=N+1}^{N+M} \sum_{i=1}^{N} x_{ijk} \leq 1 \quad (k = 1, \ldots, K); \tag{1.8}$$

Restricciones de eliminación de subtours.

$$y_i - y_j + (M + N) x_{ijk} \leq N + M - 1; \quad \text{para } 1 \leq i \neq j \leq N \text{ y } 1 \leq k \leq K; \tag{1.9}$$

Define la naturaleza binaria de las variables de decisión de ruta.

$$x_{ijk} \in \{0, 1\} \quad \forall i, j, k; \tag{1.10}$$

Define la naturaleza binaria de las variables auxiliares de eliminación de subtours.

$$y_i \in \{0, 1\} \quad \forall i; \tag{1.11}$$

En la Figura 1.1 se ilustra la estructura del problema:

- Se observan tres depósitos (representados por cuadrados), cada uno con una flota de vehículos.
- Los clientes (círculos rojos) son asignados a un depósito específico.
- Las líneas verdes muestran las rutas de los vehículos, que parten y regresan a su depósito de origen.

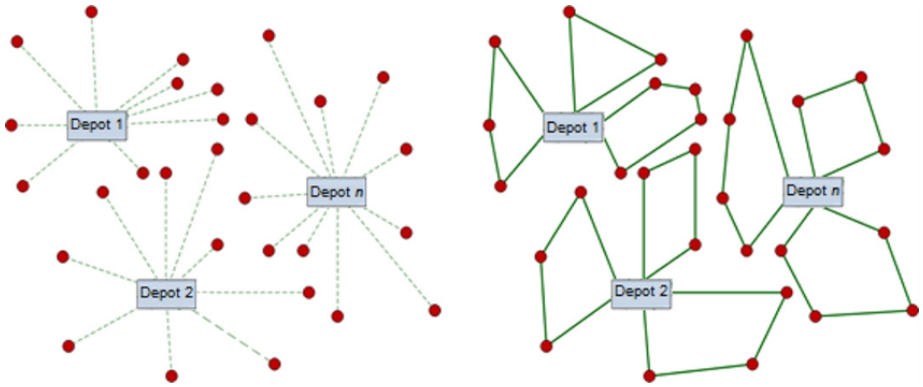


Figura 1.1: Representación gráfica de MDVRP.

La literatura documenta múltiples aplicaciones del MDVRP a problemas logísticos reales. Un estudio aplicado a una red de distribución de productos de limpieza y cuidado personal en Turquía utiliza el MDVRP para optimizar las rutas desde tres depósitos a 502 clientes. Mediante un enfoque de agrupar primero y planificar después se logran mejoras significativas en la eficiencia de la distribución, utilizando datos reales de demanda y coordenadas geográficas [35].

Otro ejemplo se encuentra en el Reino Unido, donde se desarrolla un modelo de programación entera mixta (MIP) [36] para resolver un MDVRP que incluye complejidades del mundo real

como una flota heterogénea, costos de conductores y ventanas de tiempo. La implementación de este modelo permite reducir sustancialmente los costos de transporte sin necesidad de abrir nuevas instalaciones [37].

También se encuentra un caso de estudio en Estambul que trata sobre la distribución de una empresa de bebidas utilizando datos históricos de GPS para construir una matriz de tiempo-distancia más precisa. Se evalúan múltiples enfoques de optimización, incluyendo algoritmos evolutivos y de aprendizaje profundo, para resolver el MDVRP. Los resultados muestran que estos métodos son efectivos para manejar la complejidad y la naturaleza dinámica de las entregas en entornos urbanos de alta densidad [38].

1.2 Algoritmos heurísticos

Debido a la complejidad que presentan los VRP, existen dos tipos de algoritmos que pueden resolver este tipo de problemas, los métodos exactos y aproximados. Cada uno de ellos posee diferentes características que pueden considerarse ventajas y/o desventajas que deben ser tomadas en cuenta al ser utilizados [28]. En la Figura 1.2 se puede apreciar una taxonomía de estos métodos:

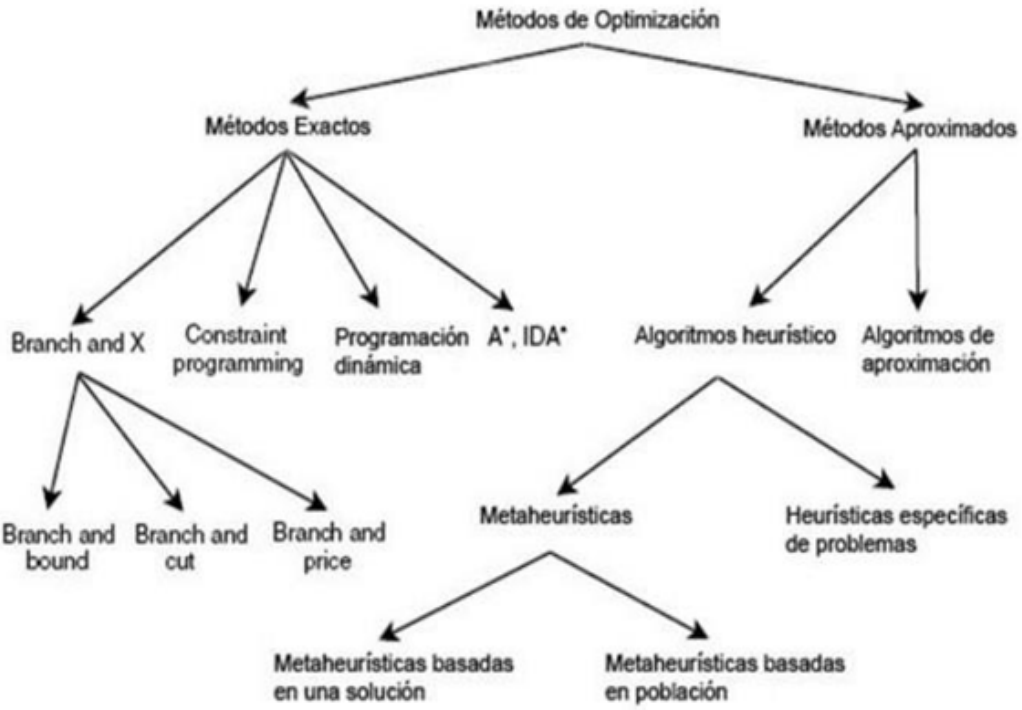


Figura 1.2: Técnicas utilizadas para la solución de MDVRP [39].

Entre los métodos exactos se pueden distinguir tres grupos: búsqueda directa de árbol, programación dinámica y programación lineal, entera y mixta. Todos estos métodos son eficientes en problemas de hasta 50 clientes, debido a restricciones de tiempo computacional. Se han llevado a cabo algunas investigaciones introduciendo métodos exactos para MDVRP [28]. Algunos ejemplos de los más utilizados para el MDVRP de este tipo de métodos son:

- Ramificación y Corte: los algoritmos *Branch & Cut* se basan en el cálculo simultáneo de cotas inferiores y superiores de la solución óptima. Las cotas superiores se obtienen por medio de la resolución de las relajaciones lineales asociadas a cada nodo del árbol de enumeración, eventualmente ajustados por medio de planos de corte, mientras que las cotas inferiores son halladas por medio de procedimientos heurísticos y cuando la relajación lineal asociada con algún subproblema tiene óptimo entero [40].
- Programación Lineal: la programación lineal es un método a través del cual se optimiza una función objetivo, bien sea maximizando o minimizando dicha función, en la cual las

variables están elevadas a la potencia 1. Este campo de la programación lineal es un área de la programación matemática, dedicada a maximizar o minimizar una función lineal que recibe el nombre de "función objetivo", de manera que las variables de tal función estén sujetas a una serie de restricciones expresadas a través de un sistema de ecuaciones o inecuaciones lineales [28].

La investigación se centra en los métodos heurísticos, los cuales se han caracterizado por ser procedimientos que garantizan buenas soluciones mediante una exploración limitada del espacio de búsqueda. Los métodos heurísticos se pueden clasificar en métodos constructivos y heurísticas de mejora [28]. Algunas heurísticas utilizadas en la literatura para solucionar el MDVRP son:

- Heurística ad hoc: en 1993 fue presentada una nueva heurística ad hoc en el aporte de [41], su método de solución asigna los clientes a los depósitos de manera fácil y rápida. En el procedimiento de mejora, permite que la distancia total recorrida por la flota aumente con la esperanza de encontrar finalmente rutas muy cortas [28].

- Algoritmo de Ahorros: extensión del método clásico para VRP al contexto de múltiples depósitos. Asigna clientes a depósitos y luego construye rutas fusionando recorridos según el ahorro de distancia [3]. En [42] se presentó la primera adaptación para MDVRP.

- Algoritmo de Barrido: método de dos fases: primero agrupa clientes por ángulo polar respecto al depósito, luego construye rutas para cada grupo. Bueno para problemas de tamaño medio-grande [43].

- *Merge-Head and Drop-Tail*: una técnica planteada para resolver el MDVRP es la denominada estrategia *Merge-Head and Drop-Tail* (MHDT), combinada con optimización de combinación de sectores (SCO), aplicada por [44]; aquí se plantea un comportamiento inverso a la entrega, es decir debe generar rutas de recolección, dada esta única diferencia, el resto de las características y restricciones son idénticas, su propuesta funciona de igual manera para el MDVRP. En este método, primero asignan puntos de recolección de acuerdo con

la distancia más cercana a los puntos de recolección, luego cada planta resuelve el VRP respectivamente, asignando clientes a los vehículos y planificando el orden en que los vehículos visitan a los clientes. En el último paso, proponen el algoritmo de optimización de combinación de sectores (SCO) para generar múltiples soluciones iniciales, y luego estas soluciones iniciales se mejoran utilizando la estrategia de combinación de cabeza y caída (MHDT).

Las metaheurísticas son estrategias de alto nivel que guían y modifican heurísticas subyacentes para explorar el espacio de búsqueda de forma eficiente. Estos algoritmos se basan en la exploración iterativa del espacio de soluciones, utilizando estrategias para escapar de los óptimos locales. Estas se clasifican en basadas en población o basadas en un punto [45].

La Búsqueda Tabú se mantiene como uno de los mejores enfoques para resolver el MDVRP. Esta es una metaheurística que explora el espacio de soluciones permitiendo movimientos que empeoran la solución actual para escapar de óptimos locales. Para evitar ciclos, mantiene una lista de movimientos prohibidos o tabú durante un número determinado de iteraciones [40].

También otra de las metaheurísticas de punto único que se destaca es la Búsqueda Local Iterada. Este algoritmo parte de una solución inicial, aplica una perturbación para diversificar la búsqueda, luego una búsqueda local para alcanzar un nuevo óptimo y finalmente decide si acepta la nueva solución según algún criterio [28].

Entre los poblacionales se destacan bastante los genéticos por la flexibilidad ante ciertas restricciones. Estos algoritmos mantienen una población de soluciones que evoluciona mediante operadores de selección, cruce y mutación. Para el MDVRP, las soluciones se representan como secuencias de clientes. El cruce combina sub-rutas de dos padres para generar un descendiente, reparando la factibilidad si es necesario [40].

Otro de los algoritmos poblacionales que se destacan fue el Algoritmo Colonia de Hormigas. Esta simula el comportamiento de hormigas reales que depositan feromona en el camino. Las hormigas artificiales construyen soluciones probabilísticamente guiadas por la feromona (que

refleja la calidad de soluciones previas) y por información heurística local. Para MDVRP, se optimizan rutas por depósito y se añaden operadores de cruce para mejorar el rendimiento [28].

1.3 Algoritmos de agrupamiento para la fase de asignación de clientes a depósitos

La minería de datos es un campo interdisciplinario de la computación que se centra en el descubrimiento de patrones significativos en grandes volúmenes de datos mediante el uso de métodos avanzados de inteligencia artificial, aprendizaje automático, estadística y bases de datos. Su objetivo principal es extraer conocimiento valioso de los datos y transformarlo en una estructura comprensible, ya sea para su análisis o para su almacenamiento en bases de datos con miras a usos posteriores [46].

La minería de datos se basa en una variedad de técnicas y métodos que permiten extraer información útil de grandes volúmenes de datos. Algunas de las técnicas más comunes incluyen [47]:

- Clasificación: es una técnica utilizada para asignar elementos de un conjunto de datos a categorías predefinidas. Esto se logra mediante algoritmos de aprendizaje supervisado, como los árboles de decisión, las máquinas de vectores de soporte (SVM) y las redes neuronales.
- Agrupamiento: es una técnica no supervisada que organiza los datos en clústeres basados en la similitud entre los datos. Esta no requiere categorías predefinidas.
- Regresión: es una técnica utilizada para predecir un valor numérico basado en la relación entre variables.
- Reglas de asociación: identifican relaciones entre variables en un conjunto de datos.

La fase de asignación de clientes a depósitos en MDVRP puede considerarse un problema de agrupamiento, ya que el resultado es un conjunto de clientes agrupados por depósito. Esta interpretación sugiere un enfoque que agrupa a los clientes y luego programa los vehículos en cada grupo. En la literatura el MDVRP puede resolverse en dos etapas: primero, los clientes deben asignarse a los depósitos; luego, se deben construir rutas que vinculen a los clientes asignados al mismo depósito. Un enfoque razonable sería dividir el problema en tantos subproblemas como depósitos existan y resolver cada subproblema por separado, lo cual transforma el MDVRP en múltiples VRP independientes [48]. Por tanto, resulta interesante adaptar los algoritmos de agrupamiento para la fase de asignación en el MDVRP.

Los algoritmos de agrupamiento se basan en la cuantificación de qué tan similares o no sean los objetos del conjunto de datos. Las métricas de distancia desempeñan un papel central en la agrupación de estos objetos en grupos. Entre las medidas comunes de similitud y distancia se incluyen varios tipos de distancia como *Euclidian*, *Manhattan*, *Haversine* y *Chebyshev*. La elección de la medida de proximidad influye en los resultados de la agrupación, ya que los distintos conjuntos de datos requieren técnicas de separación específicas y métricas de similitud adaptadas a sus características. Estos algoritmos de agrupamiento según [49], se pueden clasificar como:

- Agrupamiento particional: en estos algoritmos cada partición representa un grupo y cada objeto debe pertenecer exactamente a un grupo. Si se aplica la partición difusa, permite que los objetos pertenezcan a múltiples grupos con probabilidades. Aquí se encuentran algoritmos como k-means, k-medoids, k-modes y c-means difusos. Estos métodos son sensibles a la inicialización y a los valores atípicos, y se obtienen resultados confiables cuando los grupos son esféricos, de tamaño y distribución uniformes. En la Figura 1.3 se puede apreciar gráficamente el funcionamiento de este tipo de agrupamientos.
- Agrupamiento jerárquico: se dividen en enfoques basados en aglomeración (de abajo hacia arriba) y divisivos (de arriba hacia abajo), que producen una jerarquía de grupos

representados como dendrogramas. Los métodos basados en aglomeración comienzan con cada punto de datos como un grupo separado y fusionan los grupos más cercanos de forma iterativa, mientras que los métodos divisivos comienzan con todos los puntos de datos en un grupo y los dividen de forma recursiva. Los criterios de vinculación incluyen la vinculación simple, completa, promedio y el método de Ward, cada uno de los cuales define la proximidad del grupo de manera diferente. Los ejemplos de algoritmos de agrupamiento jerárquico incluyen BIRCH, CURE y ROCK. En la Figura 1.4 se puede apreciar gráficamente el funcionamiento de este tipo de agrupamientos.

- Agrupamiento basado en densidad: los algoritmos de agrupamiento basados en densidad identifican los agrupamientos como regiones de alta densidad separadas por áreas de baja densidad, sin requerir el número de agrupamientos como entrada. Dentro de esta clasificación se encuentra el agrupamiento espacial basado en densidad de aplicaciones con ruido (DBSCAN), el agrupamiento de puntos para identificar la estructura del agrupamiento (OPTICS), el agrupamiento de picos de densidad (DPC) y el agrupamiento jerárquico basado en densidad (HDBSCAN), este último destaca por construir una jerarquía de clústeres que le permite manejar densidades variables sin requerir un parámetro global de radio. Estos métodos pueden presentar dificultades con densidades variables. En la Figura 1.5 se puede apreciar gráficamente el funcionamiento de este tipo de agrupamientos.

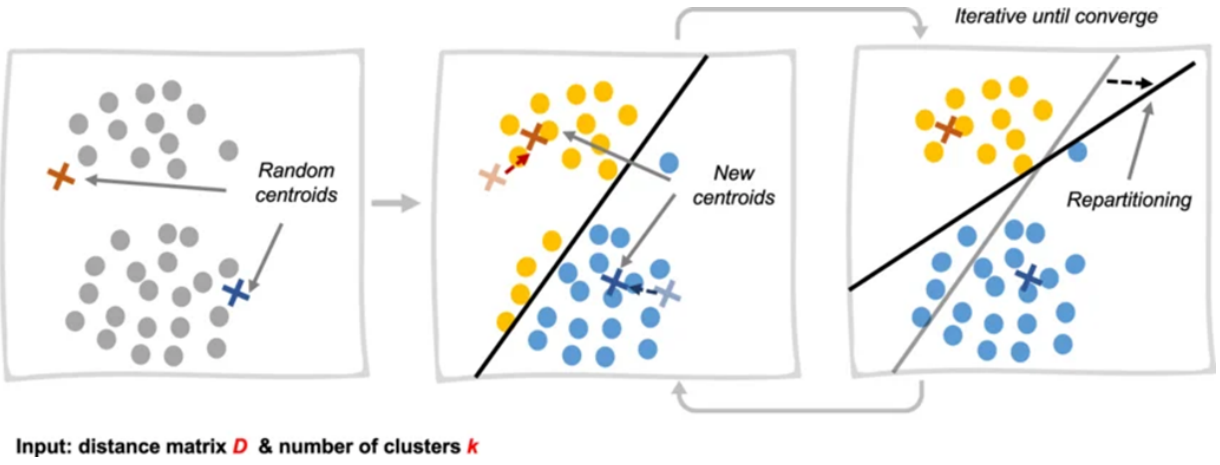


Figura 1.3: Representación gráfica de algoritmos de agrupamiento particionales [50].

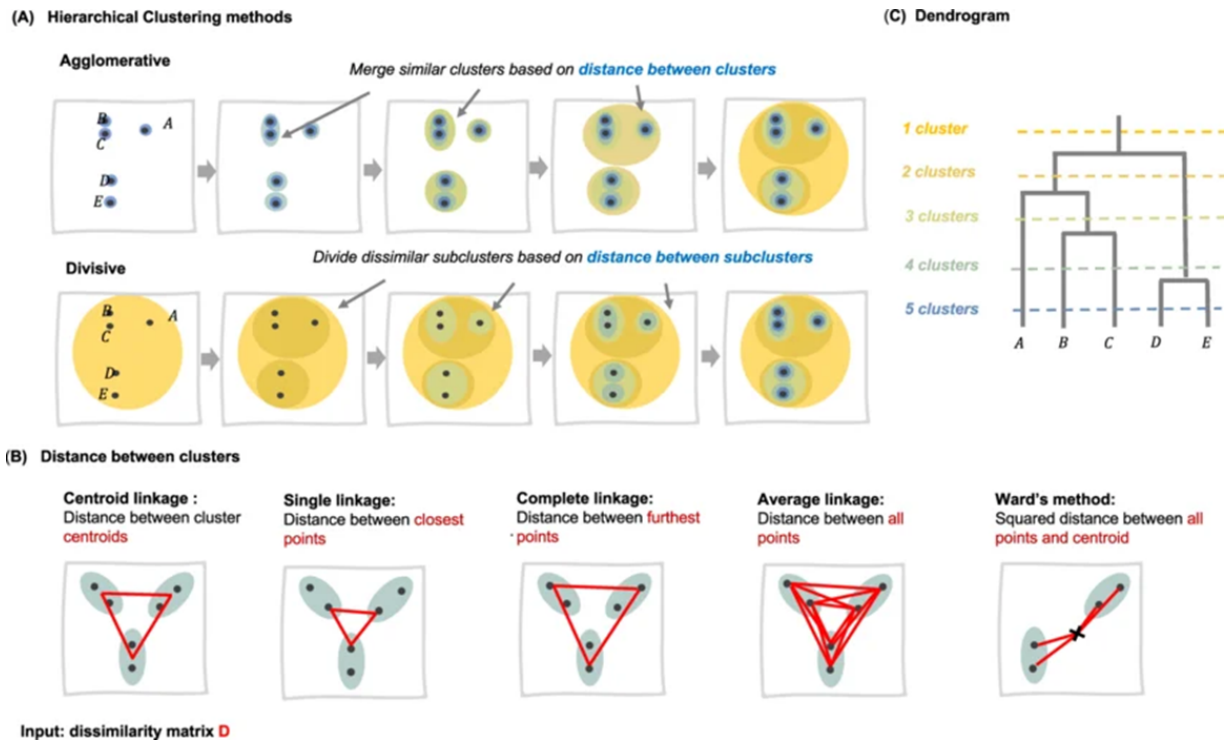


Figura 1.4: Representación gráfica de algoritmos de agrupamiento jerárquicos [50].

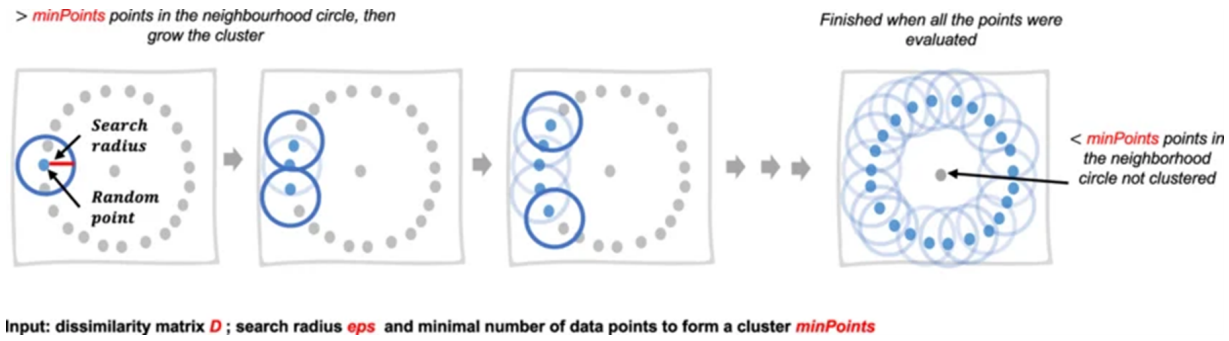


Figura 1.5: Representación gráfica de algoritmos de agrupamiento basados en densidad [50].

Los algoritmos de agrupamiento se aplican al MDVRP principalmente bajo el enfoque "agrupar primero, planificar después", donde primero se agrupan clientes y luego se construyen rutas para cada grupo. Un buen ejemplo de la aplicación de esto es una distribución de alimentos a supermercados en Reino Unido donde se utilizó K-means para agrupar 120 supermercados en cuatro clústeres correspondientes a los depósitos disponibles, minimizando la distancia intra-cluster. Luego se optimizaron rutas independientes para cada clúster. El ahorro en costos operativos fue del 15 % [51].

1.4 Biblioteca de Heurísticas de Asignación en la Planificación de Rutas de Vehículos con Múltiples Depósitos

El área de optimización ha llevado a la creación de diversas bibliotecas en diferentes lenguajes de programación, cada una con su propia variedad de algoritmos y características. A continuación, se presenta una lista no exhaustiva de bibliotecas destacadas en resolver VRP y sus variantes, entre las cuales se incluye el MDVRP. De cada una se ofrece una breve descripción de sus características y usos [17]:

- JSprit: es una biblioteca de código abierto escrita en Java. Es un componente ligero y flexible que resulta fácil de utilizar, fundamentado en una única metaheurística multipropósito que actualmente aborda una variedad amplia de problemas, como el Problema del Agente Viajero y sus variantes de VRP. Aunque no está específicamente diseñada para abordar la fase de asignación en problemas con múltiples depósitos, JSprit puede implementar estrategias de agrupamiento geográfico como un paso preliminar, utilizando técnicas como ordenamientos angulares o agrupaciones espaciales similares al enfoque del clásico algoritmo de barrido. Estas estrategias permiten organizar a los clientes en función de su proximidad antes de aplicar su metaheurística interna [52].

- *Google Operation Research Tools* (OR-Tools): es un software de código abierto desarrollado en C++, con versiones estables disponibles en Java, C# y Python. Originalmente diseñado para abordar diversos problemas de optimización como la programación lineal, la programación entera y la programación con restricciones, esta herramienta ha ido incorporando gradualmente capacidades para resolver variantes del VRP. Aunque brinda herramientas para construir modelos completos de VRP, su arquitectura flexible permite personalizar fases específicas, como la asignación, mediante programación y heurísticas personalizadas. Además, se pueden integrar algoritmos de agrupamiento para realizar una asignación previa de clientes a los depósitos. También es posible implementar restricciones a medida en el modelo,

como límites de capacidad o balanceo de carga, para ajustar la asignación inicial antes de continuar con la planificación de las rutas de los vehículos. Esta herramienta resuelve varias variantes de VRP de manera nativa, pero no el MDVRP [53].

- *Vehicle Routing Open-Source Optimization Machine* (VROOM): es un componente de software para problemas de planificación de rutas de vehículos desarrollado en el lenguaje de programación C++. Esta herramienta utiliza *Open Street Map* (OSM) y *Open Source Routing Machine* (OSRM) como *backend* para obtener rutas y devolver soluciones para las diferentes variantes del VRP. VROOM puede resolver distintas variantes del VRP que incluyen múltiples depósitos, capacidades, ventanas de tiempo, así como entrega y recogida [54].

- Scikit-learn: es una biblioteca de aprendizaje automático en Python que, aunque no está diseñada específicamente para resolver problemas de optimización como el MDVRP, proporciona una amplia gama de algoritmos de agrupamiento y análisis de datos que pueden ser utilizados para abordar la fase de asignación en este tipo de problemas. Algoritmos como K-means, DBSCAN y el agrupamiento jerárquico son útiles para preprocesar los datos y asignar clientes a depósitos de manera eficiente, basándose en proximidad y restricciones de capacidad. Su flexibilidad y compatibilidad con otras bibliotecas de Python la convierten en una herramienta versátil para integrar técnicas de agrupamiento en soluciones personalizadas para el MDVRP [55].

En el ámbito nacional, se han desarrollado diversas soluciones en el contexto académico orientadas a la resolución de problemas de planificación de rutas de vehículos, como parte del proyecto de Optimización y Metaheurísticas de la Facultad de Ingeniería Informática de la Universidad Tecnológica de La Habana “José Antonio Echeverría” (CUJAE). Algunas de estas herramientas se caracterizan por su enfoque modular y por estar diseñadas para abordar distintas variantes del VRP, con especial énfasis en la utilización de heurísticas y metaheurísticas aplicadas al MDVRP [17]:

- Bibliotecas de Heurísticas de Construcción para Problemas de Planificación de Rutas de Vehículos (BHCVRP): es un componente que implementa diez heurísticas de construcción para resolver siete variantes del VRP, incluyendo el MDVRP. Actualmente las soluciones obtenidas mediante estas heurísticas pueden ser utilizadas como solución final o como parte de una solución más compleja en un contexto donde se empleen metaheurísticas [56].

- Capa de servicio para soluciones a Problemas de Planificación de Rutas de Vehículos (CaSVRP): es una capa de servicios web dirigidos a la resolución de diferentes variantes de VRP mediante algoritmos metaheurísticos. Entre las variantes del VRP que resuelve se halla MDVRP [57]. Esta solución se compone de un módulo de optimización donde se utiliza la Biblioteca de Clases para la Integración de Algoritmos Metaheurísticos (BiCIAM)[58], un módulo para la generación de soluciones iniciales empleando BHCVRP y otro para la generación de nuevas soluciones a partir de otras soluciones haciendo uso de LMOVRP [59].

- Biblioteca de Heurísticas de Asignación para Problemas de Planificación de Rutas de Vehículos con Múltiples Depósitos (BHAVRP): es una biblioteca que resuelve la fase de asignación de clientes a depósitos en el MDVRP. Esta herramienta contiene 20 algoritmos de agrupamiento y heurísticas clásicas de asignación, y es utilizada por otras herramientas como BHCVRP, CaSVRP y TransO [17].

La Tabla 1.1 presenta una comparación entre las bibliotecas antes mencionadas que resuelven la fase de asignación del MDVRP. Es importante resaltar criterios como el hecho de ser de código abierto ya que eso sería de utilidad para seguir mejorando el componente.

Tabla 1.1: Comparación entre las herramientas que resuelven el MDVRP.

Biblioteca	Implementan heurísticas clásicas	Implementan algoritmos de agrupamiento	Lenguajes soportados	Código abierto
JSprit	No	No	Java	Sí
OR-Tools	Sí	No	C++, Python, Java, C#	Sí
VROOM	No	No	C++	No
Scikit-learn	No	Sí	Python	Sí
CaSVRP	Sí	No	Java	Sí
BHCVRP	Sí	No	Java, Python	Sí
BHAVRP	Sí	Sí	Java, Python	Sí

A partir del análisis comparativo de las bibliotecas podemos ver que solo BHAVRP y Scikit-learn ofrecen algoritmos de agrupamiento, lo que los posiciona como herramientas especialmente útiles cuando las necesidades incluyen esta capacidad para el preprocesamiento o modelado del problema. BHAVRP es la única biblioteca específicamente enfocada en resolver la fase de asignación del MDVRP, mediante la implementación de heurísticas clásicas y algoritmos de agrupamiento. Las demás bibliotecas analizadas abordan el problema completo o incluyen la asignación como un paso preliminar, sin especialización explícita en esta etapa.

BHAVRP actualmente ha sido integrado como parte funcional en herramientas como: CaSVRP y BHCVRP. Aunque su uso se ha concentrado en estos sistemas, su arquitectura modular y sus implementaciones en Java y Python permiten su potencial integración en otras aplicaciones compatibles con estos entornos [17].

Como se puede apreciar en la Figura 1.6, el flujo de ejecución comienza en el software que consume BHAVRP, donde se introducen los datos correspondientes a una instancia del problema MDVRP. Estos datos se envían al componente BHAVRP, el cual procede a cargar los datos del problema y a determinar el tipo de distancia que se utilizará en el cálculo de costos. A continuación, BHAVRP verifica si los datos fueron cargados correctamente. Si los datos presentan

errores, el proceso se detiene. En caso contrario, se configuran los parámetros de ejecución necesarios, y se genera la matriz de costo correspondiente según la distancia seleccionada. Una vez generada la matriz de costos, el algoritmo se ejecuta en el contexto de BHAVRP, utilizando los parámetros definidos y la matriz de costos generada previamente. Al finalizar la ejecución del algoritmo, BHAVRP genera una solución, que es devuelta al sistema solicitante. Este último obtiene así los resultados de la asignación de clientes a depósitos, completando el ciclo de interacción [17].

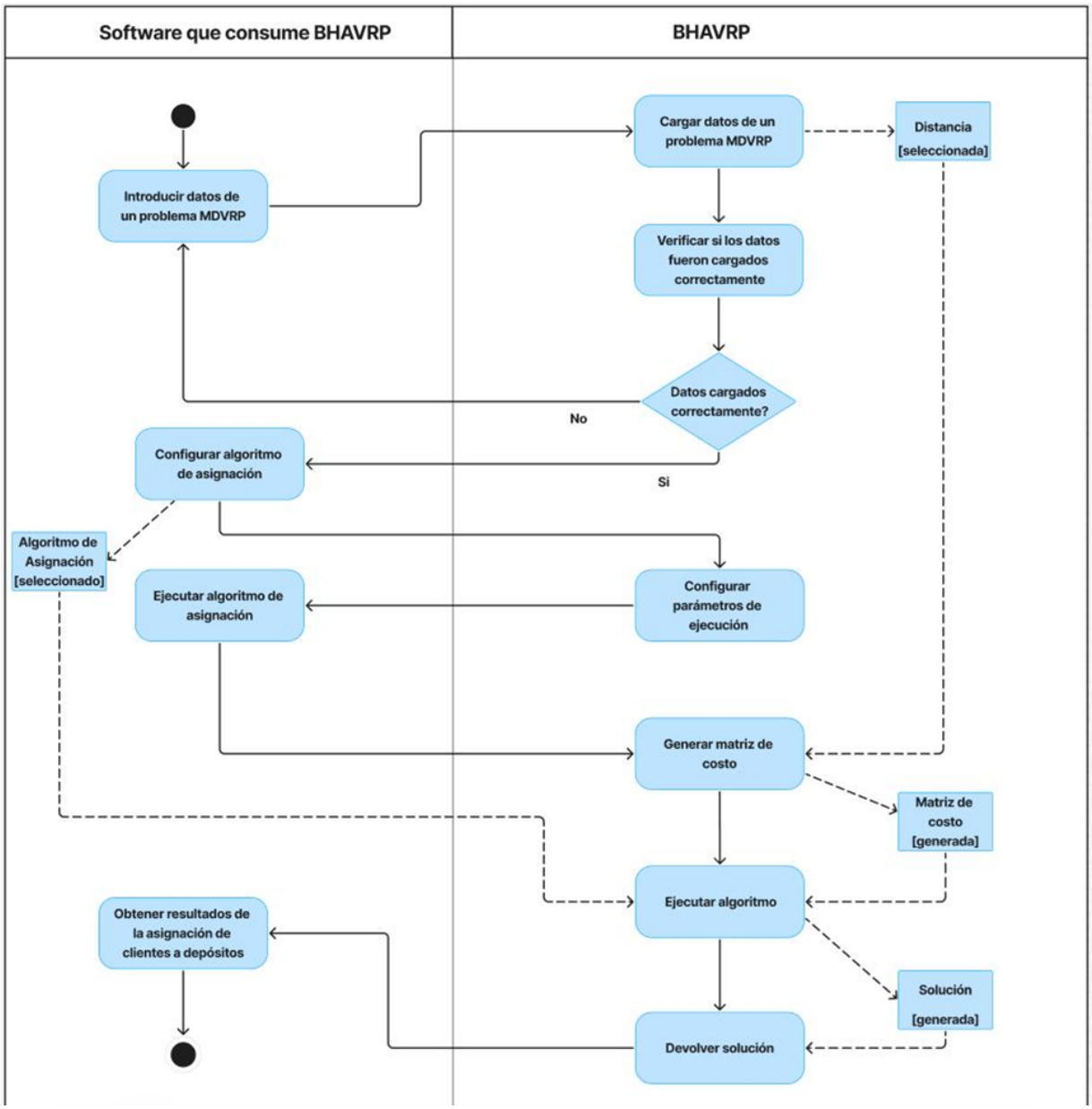


Figura 1.6: Diagrama de actividades del flujo de BHAVRP.

Para soportar la variedad de algoritmos y garantizar su correcta ejecución, BHAVRP presenta una arquitectura n-capas con un enfoque basado en reutilización, promoviendo consistencia con los lenguajes de desarrollo seleccionados [17]. Esta arquitectura se presenta en la Figura 1.7, que ofrece una vista detallada de BHAVRP.

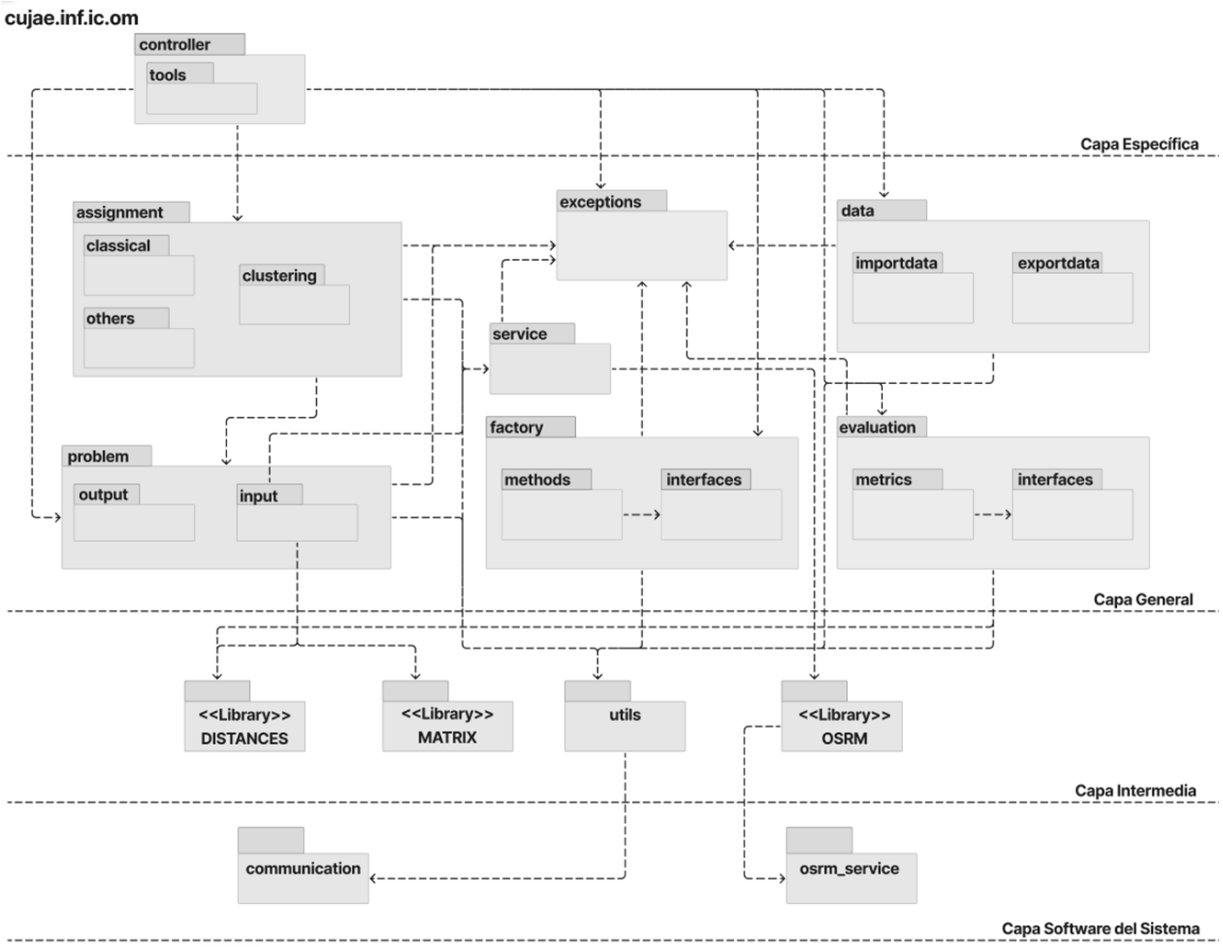


Figura 1.7: Diagrama de arquitectura n-capas con enfoque en reutilización de BHAVRP.

La capa específica contiene los elementos responsables de gestionar la ejecución del sistema, como la clase Controller, que coordina la carga de datos, la configuración de parámetros y el flujo general de ejecución. Por su parte, en la capa general, que representa el núcleo de la biblioteca es responsable de la definición del problema, la modelación de las instancias y la ejecución de los algoritmos de asignación. Por su parte, la biblioteca trabaja con una estructura de datos que representan los elementos clave del MDVRP: clientes, depósitos, y sus respectivas restricciones y parámetros. La estructura básica para utilizar la biblioteca incluye un encabezado con el número de depósitos, clientes y vehículos disponibles por depósito, seguido de las capacidades máximas de los vehículos. Los clientes y los depósitos están definidos por un identificador y sus coordenadas (x, y). Además, cada cliente tiene una demanda asociada, mientras que cada

depósito cuenta con una capacidad equivalente a la suma de las capacidades de los vehículos en su flota. BHAVRP en la capa intermedia contiene varias herramientas. Una de estas permite trabajar tanto con distancias reales como aproximadas (Euclidean, Manhattan, Chebyshev y Haversine). Luego para abordar el cálculo aproximado de las distancias, integra la herramienta OSRM (Open Source Routing Machine), que permite calcular distancias y tiempos de viaje basados en mapas reales, tanto en modalidad local como a través de solicitudes a un servidor en línea. En la capa de software se implementa la comunicación de la herramienta OSRM que expone sus servicios a través de una API HTTP RESTful. El cliente Java (como osrm-java-client) construye las peticiones HTTP y convierte las respuestas a JSON [17].

1.4.1 El problema del ruido en los algoritmos de BHAVRP

La fase de asignación de clientes a depósitos en el MDVRP se ha abordado dentro de la biblioteca principalmente mediante heurísticas clásicas de asignación y algoritmos de agrupamiento particionales. Sin embargo, estos métodos presentan la limitación de que no están diseñados para detectar ni aislar puntos atípicos. Como resultado, generan clústeres que incluyen clientes alejados del depósito asignado lo que incrementa el costo de las rutas posteriores.

Esta limitación se evidencia en los resultados experimentales reportados en [17] sobre una instancia real de 154 clientes y 5 depósitos en La Habana. La Figura 1.8 muestra la asignación generada por la heurística *Three Criteria Clustering* (TCC), donde se observa que algunos clientes asignados a un depósito se encuentran en zonas alejadas de su depósito principal. De forma similar, la Figura 1.9 presenta la asignación del algoritmo *Farthest-First* (FF), caracterizada por clústeres con más dispersión y solapamiento entre zonas de distintos depósitos.

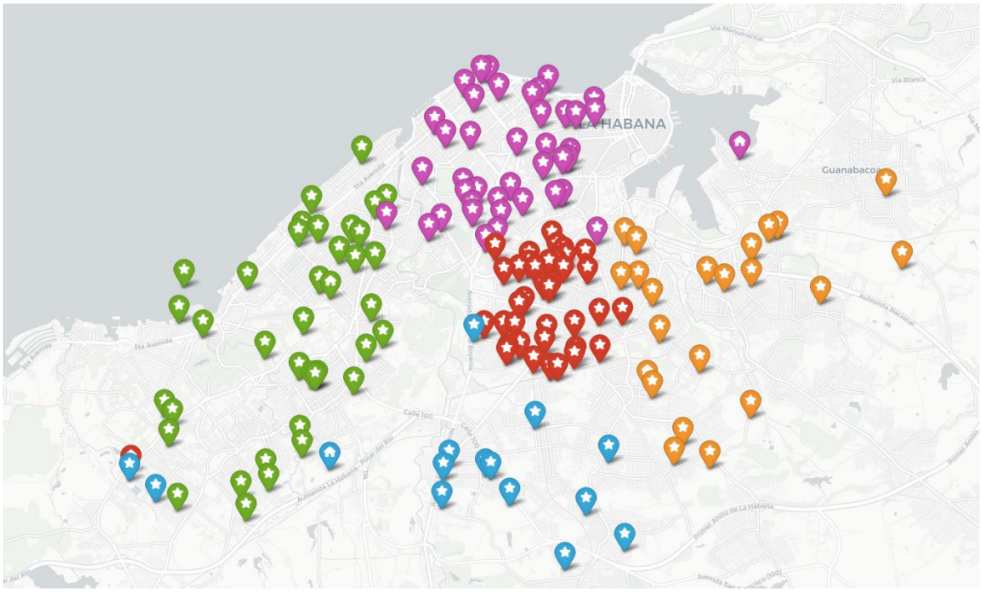


Figura 1.8: Asignación generada por el algoritmo TCC sobre una instancia real.

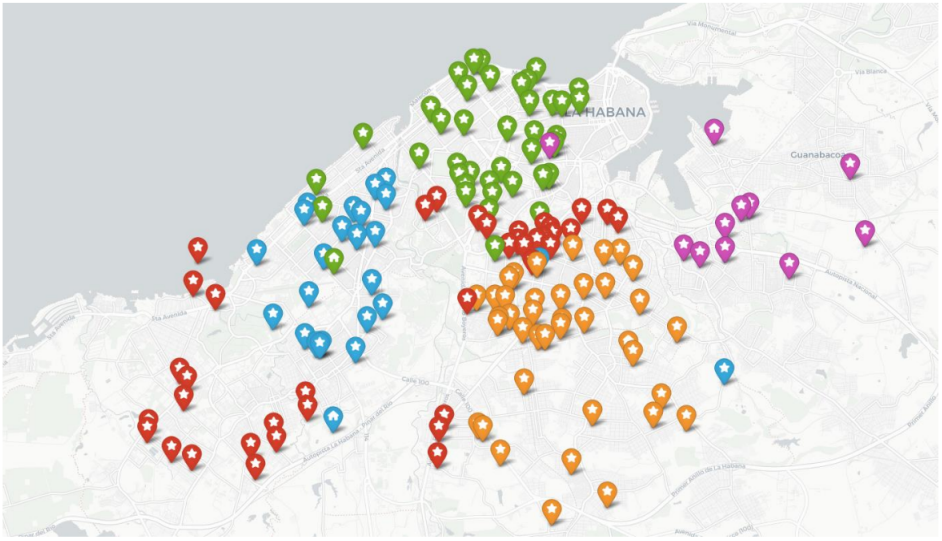


Figura 1.9: Asignación generada por el algoritmo FF sobre una instancia real.

Esto se ve reflejado en las métricas internas de validación. Algoritmos con más ruido obtienen los peores valores en métricas de cohesión y separación, lo que confirma que la presencia de este degrada la calidad estructural de la partición [17]. Adicionalmente, el análisis de correlación entre las métricas internas y el costo total de las rutas evidencia que una asignación deficiente no siempre se traduce en un costo mayor.

Estos resultados fundamentan la necesidad de incorporar a BHAVRP un algoritmo de agrupamiento basado en densidad, capaz de identificar regiones de alta concentración de clientes y aislar aquellos puntos que no pertenecen a ninguna estructura densa. A diferencia de los métodos particionales y las heurísticas clásicas, los algoritmos basados en densidad como DBSCAN, HDBSCAN u OPTICS incorporan explícitamente un mecanismo de detección de ruido, lo que permite tratarlos de forma diferenciada en la fase de asignación. Aunque la biblioteca ya contiene una adaptación de DBSCAN este presenta ciertas limitaciones a considerar, por lo que es necesario implementar otro algoritmo de esta categoría.

1.5 Conclusiones parciales

A partir del estudio realizado sobre el marco teórico de los problemas de planificación de rutas de vehículos, específicamente la variante con múltiples depósitos, así como los métodos de solución y algoritmos de Minería de Datos existentes para resolver estos problemas se puede concluir que:

- Los VRP son problemas de optimización combinatoria clasificados como NP-Duro por lo que se utilizan principalmente algoritmos heurísticos y metaheurísticos para su solución. El MDVRP es una de las numerosas variantes del VRP y requiere otro tipo de estrategias de solución.
- Una de las estrategias para la solución del MDVRP es dividir en la fase de asignación y la fase de planificación de las rutas. Además, para esta primera fase de asignación resulta efectivo el uso de algoritmos de minería de datos.
- BHAVRP es una biblioteca que permite resolver esa primera fase de asignación y tiene la facilidad para añadir nuevas heurísticas y algoritmos de agrupamiento gracias a su arquitectura n-capas orientada a reutilización.
- La incorporación de otro algoritmo basado en densidad a BHAVRP enriquece la biblioteca con otro algoritmo de esta categoría que permita tener un mecanismo de detección de ruido.

Capítulo 2: Incorporación de algoritmo a la Biblioteca de Heurísticas de Asignación del Problema de Planificación de Rutas con Múltiples Depósitos

En este capítulo se presenta la incorporación del nuevos algoritmo en BHAVRP. Se muestra una tabla comparativa entre los algoritmos de agrupamiento candidatos para implementar en la biblioteca. Además, se presenta una descripción detallada de el algoritmo de agrupamiento elegido, modificaciones en la arquitectura de la biblioteca para la incorporación de este y la adaptación para el MDVRP mediante diagrama de actividades.

2.1 Análisis y selección del algoritmo basado en densidad

La incorporación de un algoritmo basado en densidad responde a una necesidad identificada en la biblioteca BHAVRP carece de una variedad suficiente de algoritmos de esta categoría para garantizar un mecanismo detecccción de ruido en las asignaciones de clientes de depósitos. Las heuristicas clásicas y los algoritmos particionales son sensibles al ruido, mientras que los jerárquicos son más flexibles pero pueden verse afectados por el ruido [10]. Sin embargo, los algoritmos de agrupamiento basados en densidad identifican clústeres como regiones de alta densidad separadas por áreas de baja densidad siendo capaces de detectar formas arbitrarias e identificar correctamente el ruido [60].

Como candidatos a incluir en la biblioteca se seleccionaron los algoritmos de agrupamiento basados en densidad más comentados en la literatura, los cuales se presentan a continuación:

- OPTICS: genera un ordenamiento de los puntos basado en su densidad de alcanzabilidad, lo que permite identificar estructuras jerárquicas y densidades variables sin requerir un

parámetro global de radio. Sin embargo, no devuelve directamente una partición de clústeres, sino que requiere un paso posterior de extracción. Es uno de los algoritmos de referencia más citados en la literatura de agrupamiento basado en densidad [61].

- HDBSCAN: es una versión extendida de DBSCAN que construye un árbol jerárquico de clústeres basado en la distancia de alcanzabilidad mutua. Extrae los clústeres más estables, lo que le permite manejar densidades variables, detectar formas arbitrarias y aislar el ruido de forma robusta, manteniendo una complejidad computacional adecuada para grandes volúmenes de datos. Está considerado como el estado del arte dentro de los algoritmos jerárquicos basados en densidad por su enfoque híbrido de ambos enfoques [62].

- DENCLUE: modela el espacio de datos mediante funciones de influencia (como gaussianas) para estimar la densidad. Los clústeres se identifican como los máximos locales de esta función. Su rendimiento depende en gran medida de la correcta configuración del ancho de banda y del umbral de densidad. Se considera uno de los métodos pioneros en combinar funciones de densidad con técnicas de agrupamiento [63].

- DPC: identifica los centros de los clústeres buscando puntos con alta densidad local y una distancia considerable a puntos de mayor densidad, para luego asignar el resto de los puntos. Aunque es conceptualmente simple y no requiere especificar el número de clústeres, su complejidad computacional es $O(n^2)$ y carece de un mecanismo explícito para el etiquetado de ruido. Es uno de los algoritmos de agrupamiento por densidad más populares de la última década, con miles de citas y múltiples extensiones [64].

Para fundamentar la selección del algoritmo que se incluirá en BHAVRP, en la Tabla 2.1 se realizó un análisis comparativo por sus caracteristicas mas relevantes para resolver la fase de asignación del MDVRP:

Tabla 2.1: Tabla comparativa de algoritmos de agrupamiento basados en densidad.

Caracterstica	**OPTICS**	**HDBSCAN**	**DENCLUE**	**DPC**
Devuelve cluster como resultado	No	Sí	Sí	Sí
Detección de Ruido	Sí	Sí	Sí	Parcial
Pocos Parámetros	Sí	Sí	No	No
Escalabillidad	Media	Alta	Media	Baja
Maneja densidades variables	Parcial	Sí	Sí	No
Forma de los clústeres detectados	Arbitraria	Arbitraria	Arbitraria	Limitada

A partir del análisis de la Tabla 2.1, se seleccionó HDBSCAN como el algoritmo a implementar. Los criterios que fundamentan esta decisión son los siguientes:

- HDBSCAN es el único algoritmo que cumple con todos los criterios deseables de forma positiva. Devuelve clústeres como resultado, detecta ruido, requiere pocos parámetros, posee una escalabilidad alta, maneja densidades variables y detecta formas arbitrarias en los clústeres.
- Aunque OPTICS detecta ruido y requiere pocos parámetros, no devuelve clústeres como resultado directo que es un requisito muy importante para la fase de asignación del MDVRP, solo maneja densidades variables de forma parcial y su escalabilidad es media.
- Si bien DENCLUE devuelve clústeres, detecta ruido y maneja densidades variables, presenta dos desventajas clave frente a HDBSCAN, requiere muchos parámetros lo cual dificulta la selección de un parámetro global dentro de la biblioteca que permita un buen funcionamiento para todas las instancias posibles y su escalabilidad es solo media.
- El algoritmo DPC es el menos adecuado según los criterios evaluados. Requiere demasiados parámetros, su escalabilidad es baja, no maneja densidades variables, la forma de los clústeres que detecta es limitada y su detección de ruido es solo parcial.

Dado que BHAVRP ya contaba con una implementación de DBSCAN en su versión anterior, resulta interesante comparar la implementación existente con la nueva propuesta para entender las ventajas que HDBSCAN aporta a la biblioteca. La Tabla 2.2 resume las diferencias clave.

Tabla 2.2: Comparación entre DBSCAN y HDBSCAN.

Característica	**DBSCAN**	**HDBSCAN**
Parámetro ε (radio)	Requerido	No requerido
Manejo de densidades variables	No	Sí
Extracción de clústeres	Por umbral global	Por estabilidad jerárquica
Detección de ruido	Sí	Sí
Complejidad computacional	$O(n \log n)$	$O(n^2)$
Calidad de agrupamiento	Limitada por ε	Adaptativa

La principal ventaja de HDBSCAN sobre DBSCAN es la eliminación del parámetro ε, que en DBSCAN es extremadamente sensible y debe ajustarse manualmente para cada instancia. En su adaptación dentro de la biblioteca este se ajusta automaticamente, sin embargo esto puede fallar en ciertas instancias y empeorar la calidad de la solución. En HDBSCAN la densidad local se captura de forma adaptativa mediante la distancia de alcanzabilidad mutua, lo que permite descubrir clústeres de densidades variables sin intervención del usuario.

Sin embargo, esta mejora tiene un costo: HDBSCAN es computacionalmente más costoso que DBSCAN. Mientras que DBSCAN puede ejecutarse en $O(n \log n)$ con un índice espacial adecuado, HDBSCAN requiere $O(n^2)$ en su implementación actual.

La incorporación de HDBSCAN junto a DBSCAN en BHAVRP proporciona a la biblioteca dos enfoques complementarios dentro de la familia de algoritmos basados en densidad: uno muy rápido y simple con DBSCAN y otro más robusto y adaptativo con HDBSCAN. Esto enriquece las opciones disponibles para el usuario y permite tener más mecanismo para detección de ruido dentro de la biblioteca.

2.2 Algoritmo HDBSCAN

El algoritmo *Hierarchical Density-Based Spatial Clustering of Applications with Noise* (HDBSCAN) fue propuesto por [62]. El algoritmo es una extensión de DBSCAN convirtiéndolo en un híbrido entre un método jerárquico y basado en densidad, que construye un dendrograma de clústeres y luego extrae una partición plana basada en la estabilidad de los mismos. Su funcionamiento se puede descomponer en cinco fases fundamentales.

1. Transformación del espacio: el primer paso consiste en transformar el espacio de datos para reflejar la densidad local. Para cada punto p, se define su distancia al núcleo (*core distance*, $d_{core}(p)$) como la distancia al k-ésimo vecino más cercano, donde $k = \text{minPts}$. A partir de esta, se define la distancia de alcanzabilidad mutua (*Mutual Reachability Distance*, MRD) entre dos puntos p y q como:

$$d_{mrd}(p, q) = \text{máx}\{d_{core}(p), d_{core}(q), d(p, q)\} \tag{2.1}$$

   donde $d(p, q)$ es la distancia euclidiana entre p y q. Esta transformación hace que los puntos en regiones de baja densidad sean "alejados" entre sí, mientras que los puntos en regiones densas permanecen cercanos, lo que permite que el algoritmo sea robusto ante el ruido.

2. Construcción del árbol de expansión mínima: con las distancias MRD, se construye un grafo completo donde los nodos son los puntos de datos y el peso de cada arista (p, q) es $d_{mrd}(p, q)$. Sobre este grafo se calcula el Árbol de Expansión Mínima (*Minimum Spanning Tree*, MST) [10], que conecta todos los puntos con el costo total mínimo. El MST se puede construir eficientemente utilizando el algoritmo de Prim[65], con una complejidad de $O(n^2)$ en el peor caso para espacios de baja dimensionalidad.

3. Construcción de la jerarquía de clústeres: a partir del MST, se construye una jerarquía de componentes conectados. Si se ordenan las aristas del MST por peso de menor a mayor y se eliminan progresivamente, se obtiene una secuencia de componentes que se fusionan. Este proceso genera un dendrograma que representa la estructura jerárquica de los clústeres a diferentes niveles de densidad.

4. Condensación de la jerarquía: el dendrograma completo puede contener muchas ramas que representan clústeres con muy pocos puntos. Para simplificarlo, se aplica un proceso de condensación basado en el parámetro *min_cluster_size*. Las ramas que no alcanzan este tamaño mínimo se consideran ruido y se podan del árbol, dando como resultado un árbol condensado que solo contiene los clústeres significativos.

5. Extracción de clústeres estables: finalmente, se calcula la estabilidad de cada clúster en el árbol condensado. La estabilidad de un clúster C se define como la suma, sobre cada punto $p \in C$, de la diferencia entre el nivel de densidad en el que el punto se une al clúster y el nivel en el que el clúster se divide. El algoritmo selecciona el conjunto de clústeres que maximiza la estabilidad total, lo que resulta en una partición robusta que no depende de un umbral de densidad fijo.

A modo de resumen en la Figura 2.1 se puede apreciar el diagrama de flujo del algoritmo HDBSCAN base.

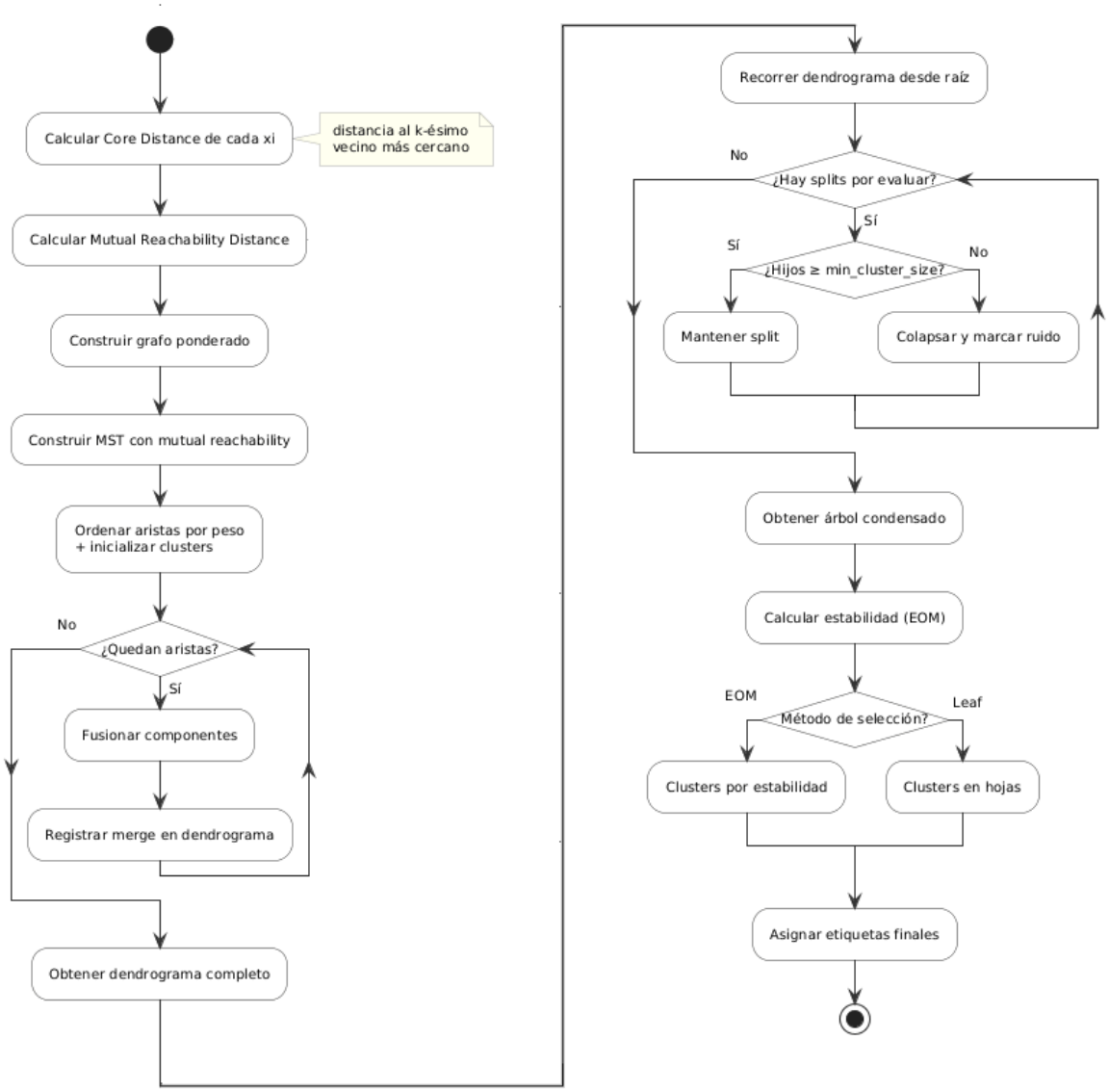


Figura 2.1: Diagrama de flujo del algoritmo HDBSCAN.

2.3 Adaptación de HDBSCAN al contexto del MDVRP

La aplicación directa de HDBSCAN al MDVRP no es posible, ya que el algoritmo original no contempla las restricciones de capacidad de los depósitos ni la necesidad de asignar cada clúster a un depósito específico. Por ello, se diseñó una adaptación que preserva la lógica de agrupamiento por densidad pura y añade una fase posterior de asignación heurística con restricciones de capacidad.

2.3.1 Mecanismo de inicialización y configuración

A diferencia de una ejecución estándar de HDBSCAN, donde los clústeres se forman libremente a partir de la densidad de los datos, en la adaptación propuesta el algoritmo opera exclusivamente sobre el conjunto de clientes. Los depósitos no se introducen en esta etapa, sino que su rol se reserva para la fase de reasignación posterior.

La clase HDBSCAN hereda de AbstractDensity, la cual proporciona métodos auxiliares para el cálculo de distancias euclidianas y la gestión de clústeres. En la fase de inicialización, se cargan las listas de clientes, depósitos y se crean las estructuras de clústeres vacías a partir de los depósitos. Además, se configuran dinámicamente los dos parámetros principales del algoritmo:

- *minPoints* (m_pts): Se calcula adaptativamente como $\max(3, \ln(1.5 \cdot N))$, donde N es el número de clientes. Esto ajusta la sensibilidad a la densidad según el tamaño de la instancia.
- *minClusterSize* (m_clSize): Se establece en 2, garantizando que cualquier agrupación significativa tenga al menos dos clientes.

2.3.2 Construcción de la jerarquía y extracción de clústeres

La implementación sigue fielmente los pasos teóricos de HDBSCAN:

1. Cálculo de distancias núcleo: para cada cliente, se calcula su distancia al m_pts-ésimo vecino más cercano (d_{core}).

2. Construcción del MST: se utiliza el algoritmo de Prim para construir el Árbol de Expansión Mínima (MST) sobre el grafo completo, donde los pesos de las aristas son las distancias de alcanzabilidad mutua (MRD), calculadas como máx$(d_{core}(p), d_{core}(q), d(p,q))$. Para optimizar la extracción del mínimo, se emplea una estructura de datos auxiliar llamada *Fibonacci Heap*, lo que reduce la complejidad de la cola de prioridad.

3. Construcción del dendrograma: se añaden aristas de auto-conexión (self-edges) con el peso de la distancia núcleo y se procesan las aristas del MST ordenadas por peso ascendente. se utiliza la estructura llamada *UnionFind* para gestionar la fusión de componentes conectados de forma eficiente, registrando el nacimiento, la muerte y la estabilidad de cada clúster candidato.

4. Extracción por estabilidad: se calcula la estabilidad $S(C_i)$ para cada clúster y se propaga hacia arriba en el árbol ($\hat{S}$). Se seleccionan aquellos clústeres que maximizan la estabilidad total, etiquetando el resto de los puntos (ruido) con el valor -1.

2.3.3 Asignación de clústeres a depósitos

Una vez que HDBSCAN ha generado los clústeres finales, se procede a la fase de asignación a los depósitos con su respectiva restricción de capacidad. El procedimiento implementado es el siguiente:

1. Cálculo del centroide: para cada clúster C_i, se calcula su centroide (cx, cy) como el promedio de las coordenadas de sus clientes.

2. Búsqueda del depósito más cercano: se calcula la distancia euclidiana entre el centroide del clúster y cada depósito, seleccionando el más cercano (d_j).

3. Verificación de capacidad: se comprueba si la demanda acumulada del clúster más la demanda actual del depósito d_j no excede su capacidad total. Si se cumple, el clúster se fusiona con el depósito y se actualiza su demanda.

4. Manejo de clústeres no asignables: si el depósito más cercano no tiene capacidad suficiente, los clientes de ese clúster se eliminan temporalmente del conjunto de clientes asignados (*assignedCustomers*). Esto permite que sean procesados individualmente en la fase de finalización, evitando que un clúster completo quede sin asignar por una restricción de capacidad.

2.3.4 Manejo del ruido y reasignación individual

Los puntos clasificados como ruido por HDBSCAN, así como los clientes de clústeres que no pudieron ser asignados por falta de capacidad, se procesan en la fase final del algoritmo. Para cada cliente no asignado se aplica la siguiente lógica:

1. Se construye una cola de prioridad con los índices de los depósitos ordenados por distancia ascendente al cliente.

2. Se recorre la cola y se asigna el cliente al primer depósito cuya capacidad restante sea suficiente para absorber su demanda individual.

3. Si ningún depósito puede absorberlo, el cliente se añade a la lista de elementos no asignados definitivamente (*unnasigned_items*) en la solución final.

A modo de resumen en la Figura 2.2 se puede apreciar el diagrama de flujo del procedimiento que realiza el algoritmo HDBSCAN adaptado para asignar los clústeres resultantes a los depósitos.

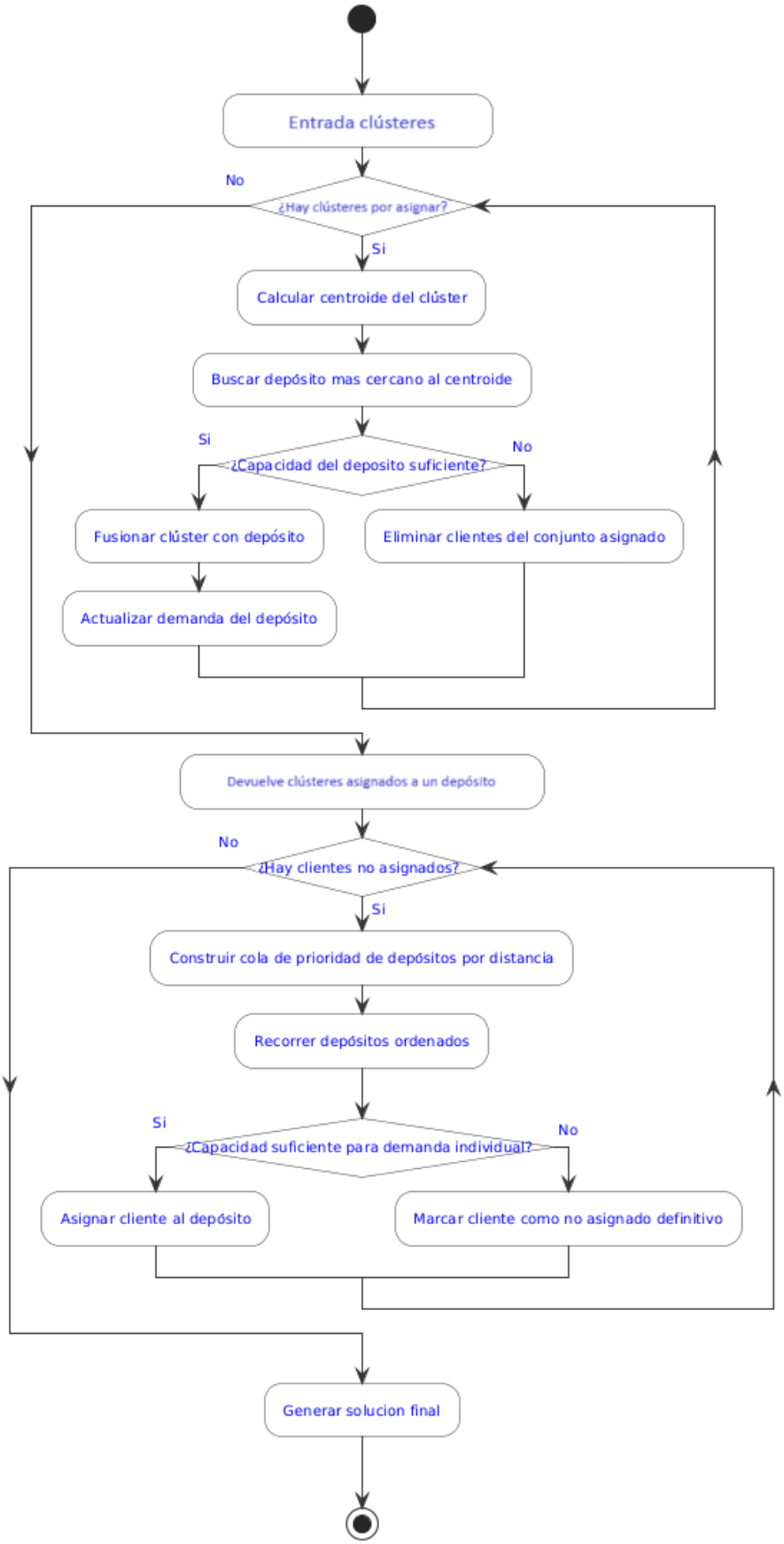


Figura 2.2: Diagrama de flujo del paso final del algoritmo HDBSCAN adaptado.

Esta versión de HDBSCAN es una adaptación al MDVRP del algoritmo disponible en la librería de código abierto *Java Statistical Analysis Tool* (JSAT)[66]. De esta librería se reutilizaron las estructuras de datos fundamentales para garantizar el correcto funcionamiento del algoritmo.

La primera estructura de datos importante es Fibonacci Heap, una estructura de cola de prioridad que ofrece un costo de $O(1)$ para las operaciones de inserción y disminución de clave (*decrease-key*), y $O(\log n)$ para la extracción del mínimo [67]. Esta característica es crucial para la construcción del Árbol de Expansión Mínima (MST) mediante el algoritmo de Prim, ya que reduce su complejidad de $O(V^2)$ a $O(E + V \log V)$ para grafos dispersos [68].

La segunda es la estructura *Disjoint Set Union* (Union-Find), utilizada para gestionar la fusión de componentes conectados durante la construcción de la jerarquía de clústeres [69]. Gracias a las optimizaciones de compresión de caminos y unión por rango, esta estructura permite que las operaciones de búsqueda y unión tengan un costo casi constante de $O(\alpha(n))$, donde α es la función inversa de Ackermann [70].

2.4 Incorporación del algoritmo propuesto a BHAVRP

La incorporación del nuevo algoritmo de agrupamiento a la biblioteca BHAVRP requirió la definición de las siguientes clases: HDBSCAN que extiende de AbstractDensity, así como un subpaquete de utilidades (*utils*) que contiene las estructuras de datos auxiliares: UnionFind, FibHeap, FibNode, Pair y Tuple3. Estas clases se integran a la arquitectura existente respetando el patrón de diseño que organiza los algoritmos de agrupamiento dentro de la biblioteca, basado en el patrón *Template Method*. En la Tabla 2.3 se puede apreciar las clases con una pequeña descripción de estas:

Tabla 2.3: Clases implementadas para la incorporación de los nuevos algoritmos de agrupamiento.

Clase	Descripción
HDBSCAN	Implementa el algoritmo HDBSCAN adaptado al MDVRP. Realiza el cálculo de distancias núcleo, la construcción del MST mediante el algoritmo de Prim, la generación del dendrograma, la extracción de clústeres por estabilidad y la asignación de estos a los depósitos respetando las capacidades.
UnionFind	Implementación de la estructura de datos *Disjoint Set Union* (Union-Find). Se utiliza durante el procesamiento de las aristas del MST para gestionar la fusión eficiente de componentes conectados y construir la jerarquía de clústeres (dendrograma).
FibHeap	Implementación de una cola de prioridad basada en un *Fibonacci Heap*. Se utiliza en el algoritmo de Prim para extraer de forma eficiente el nodo de menor distancia (peso) durante la construcción del MST.
FibNode	Representa un nodo individual dentro del FibHeap. Almacena el valor (índice del cliente), la clave (distancia), punteros a su padre, hijo, nodos hermanos y su grado, facilitando las operaciones de consolidación y disminución de clave.
Pair	Almacena un par de valores genéricos (X, Y). Se utiliza en la fase de extracción de clústeres para registrar las relaciones padre-hijo (clústeres izquierdo y derecho) en el árbol condensado.
Tuple3	Almacena una tupla de tres valores genéricos (X, Y, Z). Se emplea para representar las aristas del MST, donde X e Y son los vértices conectados y Z es el peso (distancia MRD). También se usa para ordenar las aristas durante la construcción del dendrograma.

Una vez definidas las clases que componen la solución, resulta pertinente analizar cómo interactúan durante la ejecución del algoritmo. La Figura 2.3 presenta el diagrama de secuencia de la implementación base de HDBSCAN, correspondiente a la adaptación del algoritmo original de la librería JSAT.

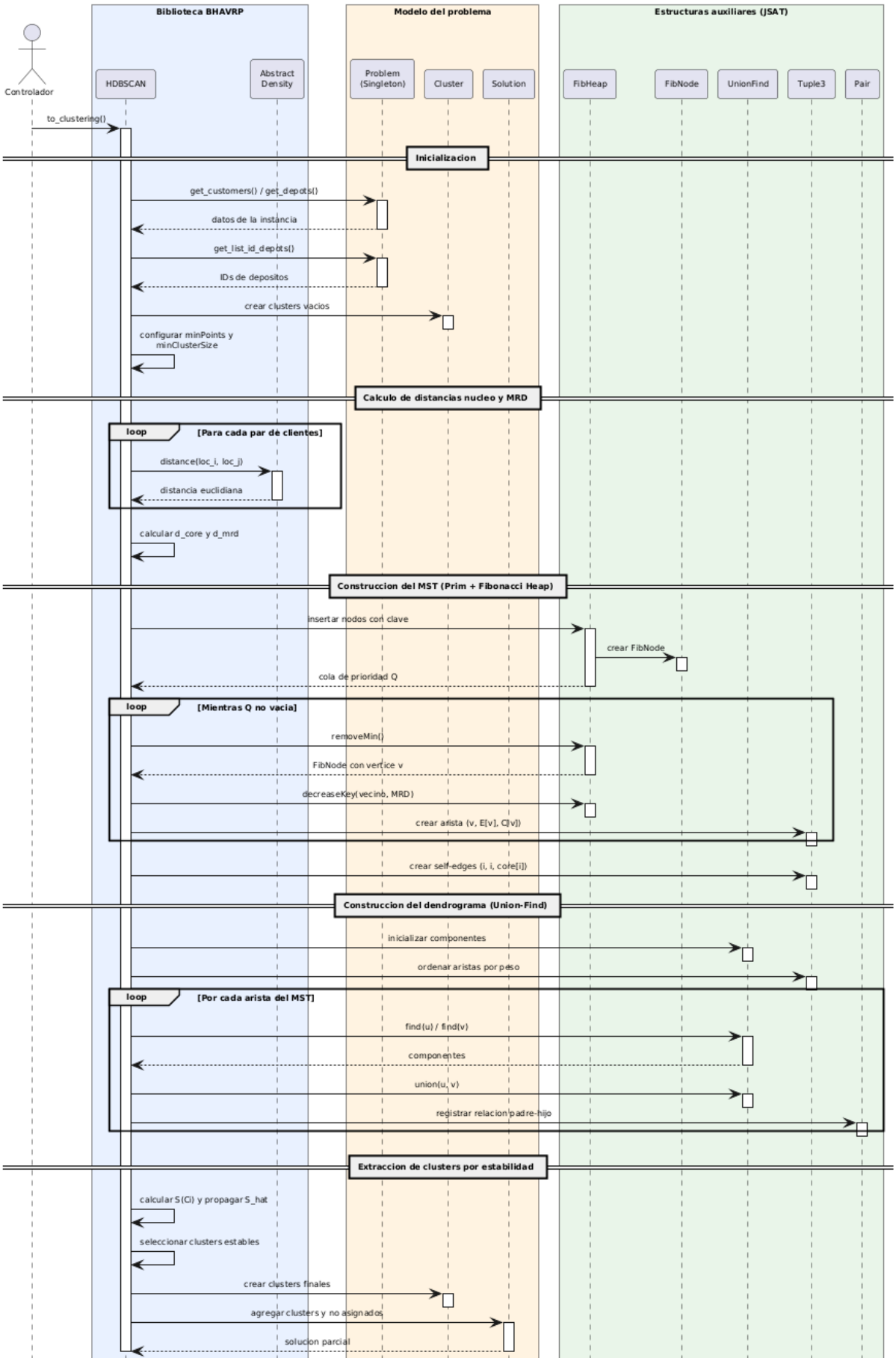


Figura 2.3: Diagrama de secuencia del algoritmo HDBSCAN.

Este diagrama de secuencia ilustra la interacción dinámica entre las clases descritas en la Tabla 2.3, abarcando desde la inicialización hasta la extracción de clústeres por estabilidad. Es importante precisar que esta figura representa principalmente el flujo de agrupamiento por densidad.

Para esta actualización de la biblioteca fue necesario modificar el paquete *Clustering* de la capa general, ya que todas estas clases fueron implementadas en este. Como se puede apreciar en la Figura 2.4, en azul se muestra el paquete modificado:

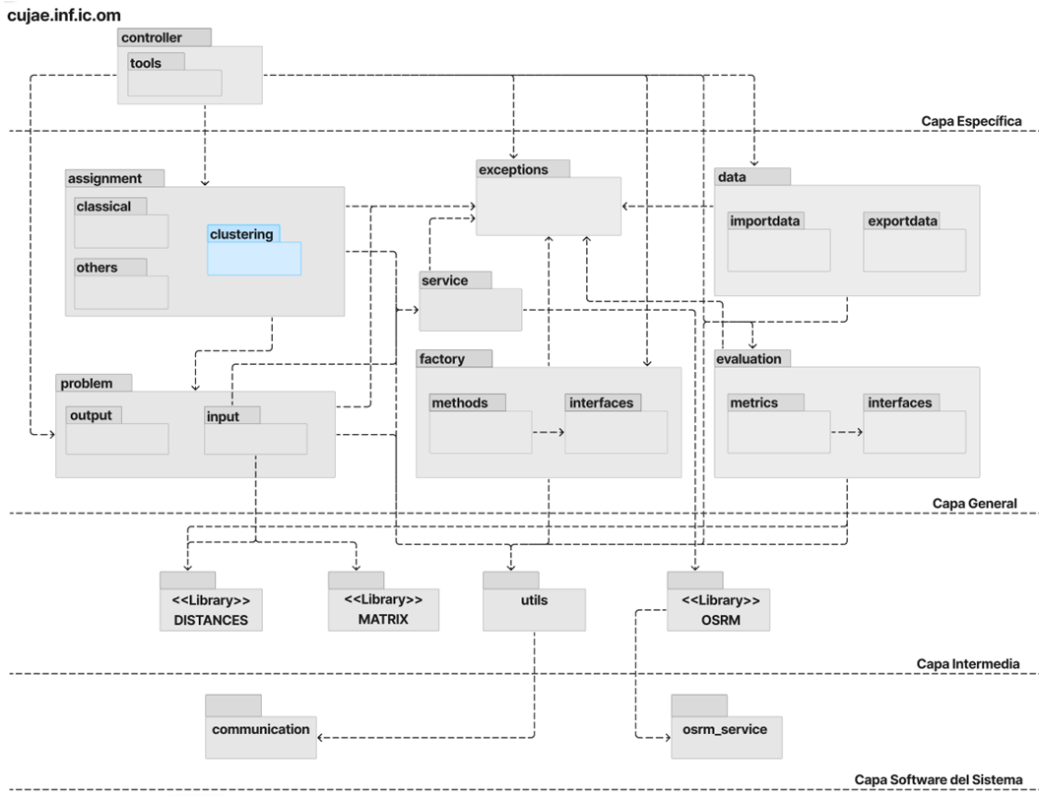


Figura 2.4: Diagrama de arquitectura actualizado.

En BHAVRP, todos los algoritmos de asignación de clientes a depósitos siguen una estructura común que consta de tres fases: inicialización, asignación iterativa y finalización. Independientemente de la heurística específica, el flujo general es siempre el mismo: se preparan las estructuras de datos, se asignan los clientes a los depósitos respetando las restricciones del problema, y se construye la solución final. Sin embargo, los detalles de cómo se realiza la inicialización, cómo

se asignan los clientes en cada iteración y cómo se determina la convergencia varían de un algoritmo a otro.

En este contexto, el patrón *Template Method* se utiliza para definir el esqueleto del algoritmo en un método plantilla que establece la secuencia invariante de ejecución. En el diseño de BHAVRP, la clase abstracta AssignmentTemplate (que hereda de la clase base Heuristic) encapsula el flujo del algoritmo en su método plantilla *toClustering* declarado en la interfaz IAssignment. Este método define la secuencia de pasos invariantes:

1. Inicialización (*initialize*): crea las estructuras necesarias, carga los datos y configura los parámetros dinámicos de HDBSCAN.

2. Asignación (*assign*): ejecuta el algoritmo HDBSCAN completo como cálculo de distancias, MST, dendrograma, estabilidad y realiza la primera fase de asignación de clústeres a depósitos.

3. Finalización (*finish*): genera la solución final, procesando los clientes no asignados como ruido y clústeres rechazados mediante la reasignación individual.

HDBSCAN respeta este patrón al implementar las operaciones abstractas definidas en la plantilla sin modificar la estructura general del flujo. El método *to_clustering* mantiene la secuencia establecida por la clase base, mientras que cada algoritmo redefine los métodos *initialize*, *assign* y *finish* para incorporar su lógica particular de agrupamiento. De esta manera, HDBSCAN implementa la expansión de clústeres basada en densidad y sigue el mismo esquema de ejecución definido por la biblioteca.

En la Figura 2.5 se puede apreciar la jerarquía de clases donde se aplica el patrón Template Method y la implementación de las nuevas clases respetando este. La clase AssignmentTemplate actúa como clase base con el método plantilla, y las distintas heurísticas de asignación sobrescriben los métodos abstractos según su lógica específica. La clase HDBSCAN extiende AbstractDensity y utiliza composición con las clases auxiliares para la construcción del MST y el dendrograma.

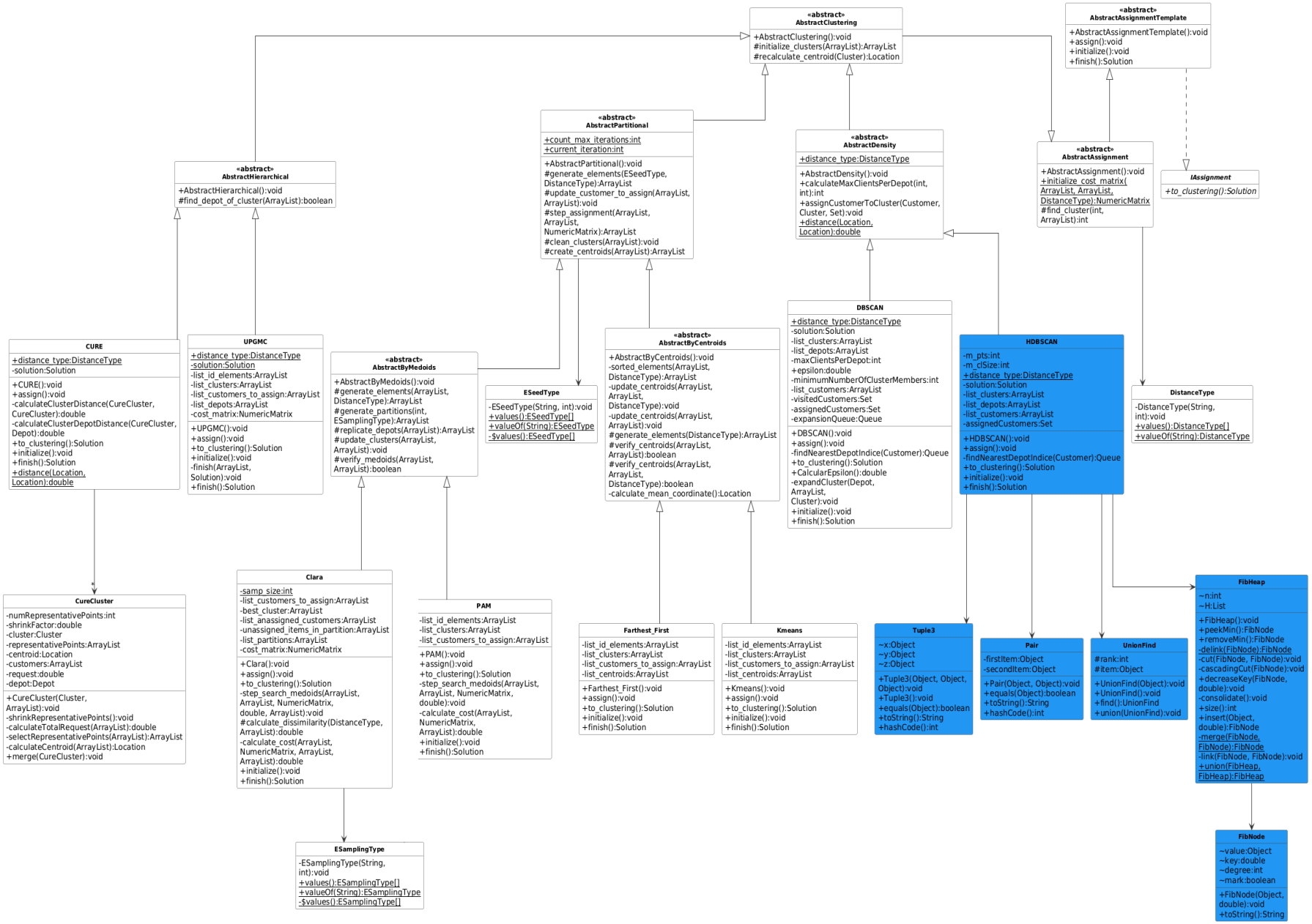


Figura 2.5: Diagrama de clases del paquete *Clustering*.

La integración de HDBSCAN respeta los siguientes principios de diseño de software:

- Principio Abierto/Cerrado: la arquitectura base de BHAVRP no requiere modificaciones para incorporar nuevos algoritmos de agrupamiento. La extensión del sistema se logra mediante la creación de nuevas clases que implementan los métodos definidos en la jerarquía existente.
- Principio de Responsabilidad Única: la clase HDBSCAN se encarga exclusivamente de la lógica de agrupamiento y asignación. Las operaciones comunes (cálculo de distancias, manejo de excepciones) se delegan a AbstractDensity, y las estructuras de datos complejas (Fibonacci Heap, Union-Find) se encapsulan en clases auxiliares independientes.
- Alta cohesión: las operaciones relacionadas con el cálculo de distancias, vecindades y otras funcionalidades compartidas se agrupan en clases específicas como AbstractDensity. Esto permite que cada componente del sistema concentre funcionalidades relacionadas entre sí.
- Bajo acoplamiento: las implementaciones concretas dependen únicamente de las interfaces y estructuras generales proporcionadas por BHAVRP, lo que facilita la sustitución o incorporación de nuevas heurísticas sin afectar el resto del sistema.

La complejidad computacional de HDBSCAN en su implementación estándar es $O(n^2)$ en el peor caso [71], debido al cálculo de la matriz de distancias mutuas y la construcción del MST sobre un grafo completo. Sin embargo, el uso de FibHeap reduce el costo de las operaciones de la cola de prioridad. La fase de asignación de clústeres a depósitos tiene una complejidad de $O(k \cdot m)$, donde k es el número de clústeres y m es el número de depósitos. Dado que $k \ll n$ y $m \ll n$, esta fase es computacionalmente despreciable en comparación con el agrupamiento.

La implementación de HDBSCAN en BHAVRP utiliza las excepciones incluidas en la biblioteca para su manejo explícito y poder para garantizar la estabilidad del algoritmo ante datos de entrada inválidos o situaciones imprevistas. Se utilizan las siguientes excepciones:

- ProblemException: se lanza cuando los datos del problema son inconsistentes, por ejemplo, lista de clientes vacía, menos de dos clientes, o ausencia de depósitos.
- ClusterException: se lanza cuando ocurre un error durante la creación o manipulación de los clústeres.
- AssignmentException: encapsula errores inesperados durante la fase de asignación, incluyendo fallos en la construcción del MST o del dendrograma.

En la fase de inicialización, se realizan validaciones como:

- Verificar que la lista de clientes no sea nula ni vacía.
- Verificar que existan al menos dos clientes para poder agrupar.
- Verificar que la lista de depósitos no esté vacía.
- Verificar que existan identificadores de depósitos para inicializar los clústeres.

Si alguna validación falla, se lanza la excepción correspondiente con un mensaje descriptivo, permitiendo que el sistema que consume BHAVRP pueda reaccionar adecuadamente y buscar una solución con mayor facilidad.

2.5 Conclusiones parciales

A partir del análisis, diseño e implementación descritos en este capítulo, se pueden extraer las siguientes conclusiones:

- La selección de HDBSCAN como algoritmo basado en densidad para BHAVRP se fundamenta en su capacidad para manejar clústeres de densidades variables y su robustez ante el ruido, superando las limitaciones de DBSCAN.
- La adaptación propuesta para el MDVRP, basada en una fase de asignación heurística por cercanía con restricciones de capacidad y reasignación individual de ruido, permite integrar el algoritmo en el contexto logístico sin comprometer la lógica de agrupamiento por densidad.
- La integración de HDBSCAN en la arquitectura de BHAVRP respeta los patrones de diseño existentes como *Template Method* y los principios de diseño de software, garantizando la mantenibilidad y extensibilidad de la biblioteca mediante el uso de clases auxiliares especializadas.

Capítulo 3: Análisis experimental

En este capítulo se presentan las pruebas funcionales y unitarias para las funcionalidades principales del algoritmo implementado, además de un estudio comparativo entre los resultados obtenidos por las heurísticas de construcción de BHCVRP usando los algoritmos de agrupamiento de la biblioteca y el nuevo algoritmo propuesto. También se realiza un análisis de correlación entre las métricas internas de BHAVRP y la calidad de las rutas obtenidas, a modo de validación complementaria.

Todos los experimentos se llevan a cabo en un entorno controlado, utilizando un ordenador con las siguientes especificaciones: procesador AMD Ryzen 5 5500U with Radeon Graphics de 2.10 GHz, 8 GB de memoria RAM y sistema operativo Windows 11 Home de 64 bits versión 25H2, usando el IDE IntelliJ IDEA con Java en su versión 23.

3.1 Pruebas funcionales

Con el propósito de garantizar la correcta integración y funcionamiento del nuevo algoritmo implementado en BHAVRP, se desarrolla un conjunto de pruebas unitarias usando el componente JUnit para validar las funciones esenciales del algoritmo HDBSCAN. Se prueban los métodos *distance* y *getNeighbours* heredados de la clase AbstractDensity y el método propio del algoritmo *findNearestDepotIndice*. Estas pruebas funcionales se realizan previamente al análisis de calidad y rendimiento del algoritmo sobre instancias.

3.1.1 Pruebas del método *distance* heredado de AbstractDensity

El método *distance* de la clase AbstractDensity es fundamental para HDBSCAN, ya que calcula la distancia euclidiana entre dos puntos geográficos representados por objetos de la clase Location. Este método se utiliza tanto en el cálculo de las distancias al núcleo como en la construcción

del árbol de expansión mínima. Para asegurar su correcto funcionamiento se diseñan pruebas unitarias que verifican:

- Cálculo correcto de distancias: se comparan distancias conocidas, como la distancia entre (0,0) y (3,4) que debe ser 5.0 por la métrica euclidiana.
- Distancia a sí mismo: se verifica que la distancia de un punto a sí mismo sea cero, lo cual es una propiedad básica de cualquier métrica.
- Manejo de coordenadas negativas: se prueba con puntos que incluyen coordenadas negativas para asegurar que el cálculo no se ve afectado por el signo.
- Simetría: se comprueba que la distancia entre A y B sea igual a la distancia entre B y A, propiedad necesaria para cualquier función de distancia.
- Consistencia del tipo de distancia: se verifica que el tipo de distancia por defecto sea la euclidiana.

En la Figura 3.1 se puede apreciar el resultado proporcionado por el IDE a esta prueba.

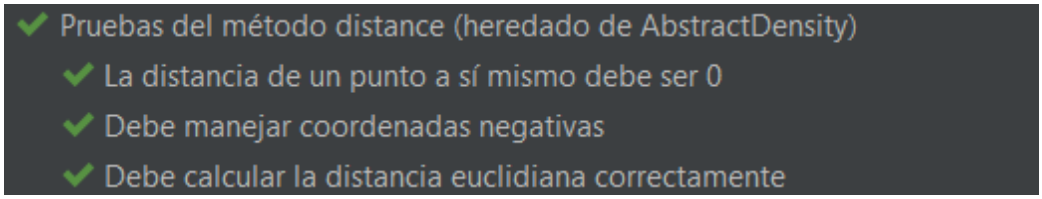


Figura 3.1: Resultados de las pruebas del método distance para HDBSCAN.

3.1.2 Pruebas del método *getNeighbours* de la clase AbstractDensity

El método *getNeighbours* es fundamental para el funcionamiento de HDBSCAN, ya que identifica los vecinos de un cliente dentro de un radio ε epsilon. Esta operación se utiliza durante el cálculo de las distancias al núcleo y en la fase de expansión de clústeres. El método recibe un cliente de entrada, la lista completa de clientes y el radio de vecindad, y retorna una lista con todos los

clientes cuya distancia al cliente de entrada es menor o igual a ε. Las pruebas unitarias diseñadas verifican:

- Vecindad básica: se comprueba que para un cliente con vecinos dentro del radio, el método retorne exactamente los clientes esperados.
- Exclusión del propio cliente: se verifica que el cliente de entrada no se incluya a sí mismo en la lista de vecinos, salvo que la implementación lo contemple explícitamente.
- Vecindad vacía: se prueba con un cliente aislado y un radio pequeño, asegurando que la lista retornada esté vacía pero no sea nula.
- Límite del radio: se comprueba que un cliente cuya distancia es exactamente igual a ε se incluya como vecino con condición $\leq$.
- Radio cero: se verifica que con $\varepsilon = 0$, solo se incluyan los clientes con coordenadas idénticas al cliente de entrada.
- Manejo de coordenadas negativas: se prueba con clientes en el segundo y tercer cuadrante del plano cartesiano.

En la Figura 3.2 se puede apreciar el resultado proporcionado por el IDE a esta prueba.

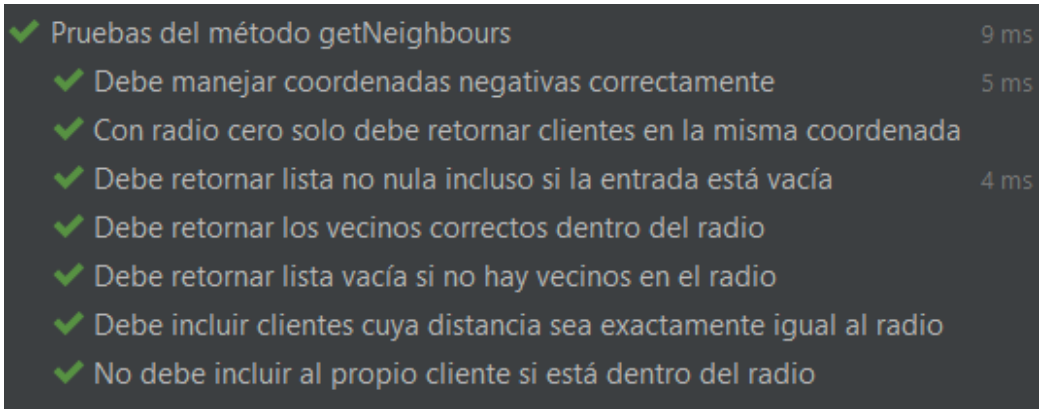


Figura 3.2: Resultados de las pruebas del método getNeighbours.

3.1.3 Pruebas del método *findNearestDepotIndice* de la clase HDBSCAN

El método *findNearestDepotIndice* construye una cola de prioridad con los índices de los depósitos ordenados de menor a mayor distancia respecto a un cliente dado. Esta operación es clave en la fase de reasignación individual, ya que permite seleccionar eficientemente el depósito más cercano con capacidad disponible. Las pruebas unitarias diseñadas verifican:

- Ordenamiento correcto: se comprueba que al extraer los elementos de la cola, los índices correspondan a los depósitos en orden ascendente de distancia al cliente.
- Tamaño de la cola: se verifica que la cola contenga exactamente tantos elementos como depósitos haya en la instancia.
- Depósito más cercano: se comprueba que el primer elemento extraído corresponda al índice del depósito más cercano al cliente.
- Depósito más lejano: se verifica que el último elemento de la cola sea el índice del depósito más alejado.
- Cliente en coordenadas de un depósito: se prueba con un cliente cuya ubicación coincide exactamente con un depósito, verificando que ese depósito sea el primero en la cola.
- Consistencia entre llamadas: se comprueba que dos llamadas consecutivas con el mismo cliente retornen colas con el mismo orden.
- Manejo de coordenadas negativas: se prueba con clientes y depósitos en cuadrantes negativos.

En la Figura 3.3 se puede apreciar el resultado proporcionado por el IDE a esta prueba.

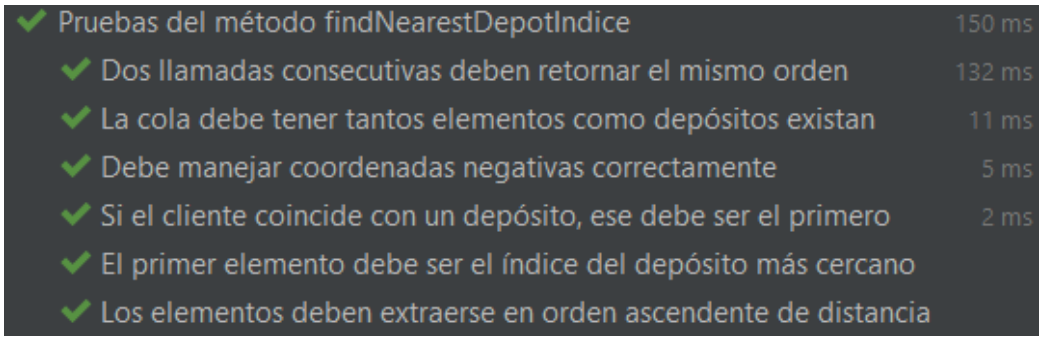


Figura 3.3: Resultados de las pruebas del método findNearestDepotIndice.

3.2 Pruebas unitarias de caja negra

Además de las pruebas funcionales de los métodos esenciales, se realizaron pruebas de caja negra para verificar el comportamiento global del algoritmo HDBSCAN en escenarios extremos, asegurando que la adaptación al MDVRP respeta las restricciones de capacidad y asignación.

En la Tabla 3.1, HDBSCAN demuestra que su mecanismo de expansión controlada desde los depósitos, junto con la reasignación final de clientes no asignados, logra respetar los límites de capacidad incluso cuando la demanda de los clústeres supera la capacidad disponible. La capacidad de HDBSCAN para identificar puntos de ruido permite tratar de forma diferenciada a los clientes atípicos, mejorando la calidad estructural de las asignaciones. Estos resultados confirman que la adaptación cumple con las reglas de negocio del MDVRP.

Tabla 3.1: Pruebas unitarias de caja negra del algoritmo HDBSCAN.

No	Descripción	Caso prueba	Juego de Datos	Resultados esperados	Resultados Reales
1	Asignación básica con dos grupos bien separados.	Asignación básica	2 depósitos, 6 clientes (3 por depósito).	Clústeres [1,2,3] y [4,5,6]. Sin no asignados.	Clústeres [1,2,3] y [4,5,6]. Sin no asignados.
2	Detección y reasignación de ruido.	Detección de ruido	7 clientes, uno aislado en (50,50).	Grupos [1,2,3] y [4,5,6]. Cliente 7 reasignado.	Grupos [1,2,3] y [4,5,6]. Cliente 7 reasignado.
3	Respeto a la capacidad de los depósitos.	Capacidad limitada	2 depósitos cap. 15, 6 clientes demanda 10 (total 60 > 30).	Se asignan hasta 30 unidades. Resto no asignados.	Se asignan hasta 30 unidades. Resto no asignados.
4	Un solo grupo denso de clientes.	Un solo clúster	5 clientes cerca del depósito 1; depósito 2 lejos.	1 clúster con 5 clientes. Depósito 2 sin asignación.	1 clúster con 5 clientes. Depósito 2 sin asignación.
5	Sin estructura de densidad clara.	Todos atípicos	5 clientes dispersos sin agrupaciones.	Finaliza sin errores. Asignación mínima o nula.	Finaliza sin errores. Asignación mínima o nula.
6	Configuración dinámica de `minPoints`.	Cálculo de `minPoints`	6 clientes.	`minPoints`=3, `minClusterSize`=2.	`minPoints`=3, `minClusterSize`=2.
7	Excepción sin problema cargado.	Error en los Datos	Sin instancia cargada.	Lanza `AssignmentException`.	Lanza `AssignmentException`.

3.3 Comparación entre los algoritmos de agrupamiento

Con el objetivo de analizar el comportamiento y la calidad de las asignaciones generadas por los algoritmos implementados en BHAVRP, se diseña un experimento enfocado en instancias utilizadas en trabajos anteriores [17, 72]. Este experimento busca comparar el rendimiento de cada algoritmo de asignación en condiciones controladas, evaluando los resultados mediante el conjunto de métricas internas incluidas dentro de la biblioteca. El estudio experimental se lleva a cabo empleando las 33 instancias originalmente introducidas por [29] en su trabajo sobre el MDVRP. Estas instancias tienen amplio reconocimiento en el ámbito científico de la optimización de rutas. La Tabla 3.2 detalla, para cada instancia del problema, el número total de clientes, la cantidad de depósitos, la flota de vehículos asignada a cada depósito y su capacidad.

Con el objetivo de fortalecer el análisis experimental, se incorporan 11 instancias adicionales, alcanzando un total de 44 instancias para la evaluación. Estas instancias provienen de la investigación realizada en [63], y se detallan en la Tabla 3.3.

Para el experimento se usaron todoa los algoritmos de BHAVRP junto con el nuevo algoritmo HDBSCAN, excluyendo RandomSequentialCyclic y SequentialCyclic ya que presentan problemas con algunas instancias al generar un solo clúster. Cada algoritmo se ejecuta una sola vez por instancia, utilizando como tipo de distancia aproximada la distancia euclidiana.

Tabla 3.2: Descripción de las instancias.

ID	Total de clientes	Total de depósitos	Total de vehículos por depósito	Capacidad de los vehículos
p01	50	4	4	80
p02	50	4	2	160
p03	75	5	3	140
p04	100	2	8	100
p05	100	2	5	200
p06	100	3	6	100
p07	100	4	4	100
p08	249	2	14	500
p09	249	3	12	500
p10	249	4	8	500
p11	249	5	6	500
p12, p13, p14	80	2	5	60
p15, p16, p17	160	4	5	60
p18, p19, p20	240	6	5	60
p21, p22, p23	360	9	5	60
pr1	48	4	1	200
pr2	96	4	2	195
pr3	144	4	3	190
pr4	192	4	4	185
pr5	240	4	5	180
pr6	288	4	6	175
pr7	72	1	6	200
pr8	144	2	6	190
pr9	216	6	3	180
pr10	288	6	4	170

Tabla 3.3: Descripción de las nuevas instancias.

ID	Total de clientes	Total de depósitos	Total de vehículos por depósito	Capacidad de los vehículos
gj1	50	2	2	492
gj2	75	3	3	415
gj3	100	2	2	841
gj4	150	4	4	651
gj5	199	5	5	758
gj6	75	2	2	669
gj7	125	3	3	768
gj8	249	4	4	1219
gj9	60	2	2	579
gj10	80	3	3	507
gj11	120	5	5	1000

3.3.1 Análisis estadístico

A partir de los experimentos realizados, se decide utilizar la prueba estadística no paramétrica de Friedman N × N [73], la cual posibilita enfrentar por pares todos los algoritmos examinados.

Para la prueba se utilizan las seis heurísticas de construcción implementadas en BHCVRP excluyendo RandomByElement debido a que presentaba errores con varias instancias. Algunas de estas heurísticas requieren de parámetros en su ejecución. A continuación, se presenta las heurística de construcción utilizadas y la configuración de estos parámetro:

- Heurística del Vecino más Cercano con Lista de Candidatos Restringidos (NN): propone la construcción de una solución insertando al cliente más cercano dentro de la lista de candidatos restringida al último cliente ya insertado. Posee como parámetro el tamaño de la lista de candidatos restringidos (size_RLC): 3

- Algoritmo de Ahorros en su versión secuencial (SaveSeq): este algoritmo plantea que en una solución dos rutas diferentes pueden ser combinadas formando una nueva ruta. Esta versión del algoritmo propone la construcción de las rutas una a una. Utiliza el parámetro λ (parameter_shape) con valor 1 para la construcción de la matriz de ahorros.

- Algoritmo de Ahorros en su versión paralela (SaveParall): esta otra versión del algoritmo tiene la misma filosofía, pero trabaja sobre todas las rutas de forma simultánea. Utiliza el parámetro λ (parameter_shape) con valor 1 para la construcción de la matriz de ahorros.

- Heurística de Inserción Secuencial de Mole & Jameson (MJ): esta heurística de inserción utiliza dos medidas para decidir el próximo cliente a insertar en la solución. Utiliza los parámetros λ (c1) con valor 1 que representa el peso del costo directo entre los clientes en la ruta, μ (c2) con valor 1 que representa el peso del costo desde el depósito hasta el cliente que se está considerando insertar y la forma de seleccionar el primer cliente a insertar en la ruta (select_customer_MJ_type) con valor 1 que significa el más cercano al depósito.

- Heurística de Barrido (Sweep): esta heurística trabaja en dos fases, primero forma los grupos de clientes y luego construyen las rutas para cada grupo.

- Heurística de Inserción en Paralelo de Christofides, Mingozzi y Toth (CMT): esta heurística de inserción opera en dos fases. En la primera fase se determina la cantidad de rutas a utilizar, junto con un cliente para inicializar cada ruta. En la segunda fase se crean dichas rutas y se inserta el resto de los clientes en ellas según el costo de inserción. Utiliza el parámetro λ (parameter_L) con valor 1, que controla el peso del costo de viajar desde el cliente j hasta el cliente i en el cálculo del costo de inserción.

En la Tabla 3.4 se presenta, para cada algoritmo de asignación, su valor promedio por cada heurística de construcción en todas las instancias. Las celdas en verde reflejan las mejores posiciones, mientras que las celdas en rojo indican los desempeños más bajos.

Tabla 3.4: Resultados de la prueba de Friedman NxN.

Algoritmo	CMT	MoleJameson	NearestNeighborWithRLC	SaveParallel	SaveSequential	Sweep
BestCyclicAssignment	12.2727	12.375	9.9545	12.2955	12.4205	10.3636
BestNearest	8.5909	6.7841	6.25	8.0341	7.8523	8.7727
CLARA	11.7727	11	12.1364	11.9545	11.9091	12.3409
CoefficientPropagation	11.9318	12.4318	10.1364	10.7045	12.0455	12.1591
CURE	6.0682	5.7841	4.5682	6.4091	6.5568	8.2045
CyclicAssignment	16.2045	15.6591	14.6591	15.8409	15.7273	15.1591
DBSCAN	11.8409	12.3182	10.7045	10.9545	12.4318	9.2386
Farthest-First	10.1136	9.4659	7.2727	10.8409	9.8864	7.9091
HDBSCAN	8.1818	8.7955	9.5	7.5795	8.7955	10.5227
KMEANS	6.1818	8.0795	5.2727	7.6477	8.1818	7.4545
NearestByCustomer	6.8636	6.6023	13.0455	7.8409	7.1818	6.8068
NearestByDepot	13.0455	11.9545	9.3636	13.3977	12.4432	11.3636
PAM	11.5682	12.4773	10.2045	11.6364	10.9773	12.6591
Parallel	7.25	6.8523	12.7273	7.8409	7.7159	6.2727
RandomByElement	19	18.9773	19	19	19	18.9318
Simplified	7.2727	7.6364	10.3182	7.75	6.6818	8.4091
Sweep	7.2045	6.1477	11.0682	6.6818	6.2614	6.5227
ThreeCriteriaClustering	7.9545	7.8295	9.4659	9.1136	7.875	8.8409
UPGMC	6.6818	8.8295	4.3523	4.4773	6.0568	8.0682

A continuación, se presenta la interpretación de los resultados obtenidos en el ranking:

- Entre los mejores algoritmos se posicionan CURE, UPGMC, KMEANS y Sweep lideran el ranking en la mayoría de las heurísticas de construcción. CURE obtiene el mejor resultado en heurística como CMT y MoleJameson, mientras que UPGMC es el mejor en NearestNeighborWithRLC, SaveParallel y SaveSequential, mientras que KMEANS y Sweep mantienen posiciones competitivas de forma consistente.
- HDBSCAN obtiene un ranking promedio que lo ubica en una posición intermedia dentro del conjunto de algoritmos evaluados. Sus mejores resultados se dan con SaveParallel y CMT, mientras que su peor desempeño ocurre con Sweep. Aunque no alcanza los primeros puestos, HDBSCAN supera consistentemente a DBSCAN en todas las heurísticas y muchas de las heurísticas clásicas de asignación, lo cual confirma la mejora que aporta respecto a su predecesor dentro de la misma familia de algoritmos basados en densidad.
- RandomByElement obtiene el peor ranking de forma sistemática, seguido por CyclicAssignment y NearestByDepot. CLARA, BestCyclicAssignment, PAM, CoefficientPropagation y DBSCAN también ocupan posiciones desfavorables.

Es importante destacar que HDBSCAN, aunque no sea de los más destacados, presenta un comportamiento sólido y consistente en los resultados de las heurísticas de construcción, con un desempeño significativamente superior al de DBSCAN y al de los algoritmos con peores resultados. Su principal fortaleza radica en la detección y manejo del ruido.

Una vez obtenidos los rankings promedio de Friedman, se aplicaron los procedimientos *post-hoc* de Bergmann [74], Holm [75] y Shaffer [76] para controlar la tasa de error familiar. La prueba de Friedman arrojó un estadístico $\chi^2 = 313.15$ con 18 grados de libertad y un valor $p = 1.55\times10^{-10}$, lo que confirma la existencia de diferencias estadísticamente significativas entre los algoritmos.

Los resultados del análisis *post-hoc* permiten extraer las siguientes conclusiones sobre el algoritmo HDBSCAN:

- Es significativamente mejor que RandomByElement con ($p = 0$), lo cual confirma que las asignaciones generadas por HDBSCAN producen rutas de mejor calidad que las del algoritmo aleatorio.
- Es significativamente mejor que DBSCAN con ($p = 0.0049$), demostrando que la evolución jerárquica basada en estabilidad supera al DBSCAN clásico en el contexto de la asignación del MDVRP.
- Es significativamente mejor que NearestByDepot con ($p = 0.000001$), superando a la heurística de asignación por depósito más cercano.
- Es significativamente mejor que PAM y CLARA con ($p = 0.00072$ y $p = 0.00027$ respectivamente), superando a los algoritmos particionales clásicos.
- UPGMC es significativamente mejor que HDBSCAN ($p = 0.0097$), lo cual es consistente con el ranking promedio.
- No se encontraron diferencias estadísticamente significativas con CURE, KMEANS, BestNearest y Sweep, lo que indica que HDBSCAN es competitivo con los mejores algoritmos en términos de calidad de rutas, aunque su promedio es ligeramente inferior.

En síntesis, HDBSCAN demuestra ser un algoritmo que aporta mejoras significativas respecto a DBSCAN y a las heurísticas clásicas, aunque no supera a los mejores algoritmos del estado de la biblioteca como CURE y UPGMC en el ranking global. Su principal valor reside en la calidad estructural de las asignaciones y en su capacidad para manejar el ruido, lo cual no siempre se traduce directamente en mejores rutas.

3.4 Análisis de correlación entre métricas internas y calidad de rutas

Con el objetivo de evaluar si existe una relación entre la calidad estructural de las asignaciones, medida mediante las métricas internas de BHAVRP que se pueden apreciar en la Tabla 3.5,

y la calidad de las rutas obtenidas en la fase posterior, se realiza un análisis de correlación complementario. A diferencia del estudio de [17], donde no se encontró correlación significativa entre la suma de las métricas normalizadas, en este estudio se analiza la relación para cada métrica y heurística de construcción por separado.

Tabla 3.5: Resumen de las métricas incorporadas en BHAVRP.

Métrica	Siglas	¿Qué evalúa?	Descripción
Suma de Errores al Cuadrado	SSE	Cohesión	Suma de las distancias cuadráticas entre cada punto y el centroide de su grupo. **Minimizar**.
Índice de Ball-Hall	BH	Cohesión	Mide la dispersión promedio dentro de los clústeres. **Minimizar**.
Suma de Cuadrados entre Clústeres	SSB	Separación	Evalúa la separación entre los grupos considerando la distancia entre sus centroides y el centro global de los datos. **Maximizar**.
Índice de Calinski-Harabasz	CH	Cohesión y Separación	Relación entre la dispersión entre clústeres (SSB) y la dispersión dentro de los clústeres (SSE). **Maximizar**.
Índice de Dunn	DI	Cohesión y Separación	Relación entre la distancia mínima inter-clúster y el diámetro máximo intra-clúster (el tamaño del grupo más grande). **Maximizar**.
Índice de Davies-Bouldin	DB	Cohesión y Separación	Analiza la relación entre la dispersión interna de los clústeres y la distancia entre sus centroides. **Minimizar**.
Coeficiente de Silueta	CS	Cohesión y Separación	Combina cohesión y separación para cada punto. **Maximizar**.
Índice de Xie-Beni	XB	Cohesión y Separación	Penaliza los agrupamientos donde los puntos están alejados de sus centroides y los que están muy próximos entre sí. **Minimizar**.
Índice de Ray-Turi	RT	Cohesión y Separación	Evalúa la relación entre la dispersión dentro de los clústeres y la distancia mínima entre los centroides de los clústeres. **Minimizar**.

Para cada heurística de construcción se calcularon los coeficientes de Spearman y Pearson entre cada métrica interna y el costo total de las rutas. Los resultados completos se presentan en las Tablas 3.6 a 3.12.

Tabla 3.6: Correlación entre métricas internas y costo total (heurística SaveParallel).

Métrica	Spearman	p-valor	Pearson	p-valor
BH	0.8555	3.02×10^{-6}	0.9161	3.68×10^{-8}
CH	-0.6921	1.02×10^{-3}	-0.7436	2.63×10^{-4}
DB	0.7577	1.72×10^{-4}	0.8459	5.06×10^{-6}
DI	-0.7893	5.88×10^{-5}	-0.8474	4.68×10^{-6}
RT	0.6816	1.31×10^{-3}	0.7578	1.71×10^{-4}
CS	-0.6605	2.08×10^{-3}	-0.7123	6.23×10^{-4}
SSB	-0.6611	2.06×10^{-3}	-0.8084	2.81×10^{-5}
SSE	0.6945	9.69×10^{-4}	0.7575	1.73×10^{-4}
XB	0.6804	1.35×10^{-3}	0.7541	1.92×10^{-4}

Tabla 3.7: Correlación entre métricas internas y costo total (heurística SaveSequential).

Métrica	Spearman	p-valor	Pearson	p-valor
BH	0.8648	1.78×10^{-6}	0.9520	3.63×10^{-10}
CH	-0.7305	3.83×10^{-4}	-0.8337	9.24×10^{-6}
DB	0.7749	9.77×10^{-5}	0.8945	2.39×10^{-7}
DI	-0.7942	4.90×10^{-5}	-0.8335	9.33×10^{-6}
RT	0.7217	4.86×10^{-4}	0.8371	7.85×10^{-6}
CS	-0.7199	5.10×10^{-4}	-0.8015	3.70×10^{-5}
SSB	-0.6292	3.90×10^{-3}	-0.8533	3.42×10^{-6}
SSE	0.7380	3.09×10^{-4}	0.8451	5.26×10^{-6}
XB	0.7222	4.79×10^{-4}	0.8348	8.79×10^{-6}

Tabla 3.8: Correlación entre métricas internas y costo total (heurística Sweep).

Métrica	Spearman	p-valor	Pearson	p-valor
BH	0.8068	2.99×10^{-5}	0.8612	2.20×10^{-6}
CH	-0.7717	1.09×10^{-4}	-0.8159	2.06×10^{-5}
DB	0.7723	1.07×10^{-4}	0.8513	3.80×10^{-6}
DI	-0.7126	6.18×10^{-4}	-0.7728	1.05×10^{-4}
RT	0.7682	1.22×10^{-4}	0.8253	1.36×10^{-5}
CS	-0.7366	3.22×10^{-4}	-0.7716	1.09×10^{-4}
SSB	-0.6617	2.03×10^{-3}	-0.8165	2.00×10^{-5}
SSE	0.7775	8.93×10^{-5}	0.8176	1.91×10^{-5}
XB	0.7723	1.07×10^{-4}	0.8237	1.47×10^{-5}

Tabla 3.9: Correlación entre métricas internas y costo total (heurística CMT).

Métrica	Spearman	p-valor	Pearson	p-valor
BH	0.9315	6.92×10^{-9}	0.9522	3.47×10^{-10}
CH	-0.8235	1.48×10^{-5}	-0.8540	3.28×10^{-6}
DB	0.8495	4.19×10^{-6}	0.9156	3.86×10^{-8}
DI	-0.8126	2.36×10^{-5}	-0.8383	7.41×10^{-6}
RT	0.8042	3.32×10^{-5}	0.8600	2.35×10^{-6}
CS	-0.7981	4.23×10^{-5}	-0.8226	1.54×10^{-5}
SSB	-0.7301	3.86×10^{-4}	-0.8829	5.59×10^{-7}
SSE	0.8284	1.18×10^{-5}	0.8624	2.05×10^{-6}
XB	0.8004	3.87×10^{-5}	0.8580	2.63×10^{-6}

Tabla 3.10: Correlación entre métricas internas y costo total (heurística MoleJameson).

Métrica	Spearman	p-valor	Pearson	p-valor
BH	0.7612	1.54×10^{-4}	0.9245	1.54×10^{-8}
CH	-0.7717	1.09×10^{-4}	-0.8569	2.81×10^{-6}
DB	0.7301	3.86×10^{-4}	0.8864	4.37×10^{-7}
DI	-0.6547	2.35×10^{-3}	-0.7569	1.76×10^{-4}
RT	0.7630	1.45×10^{-4}	0.8548	3.14×10^{-6}
CS	-0.7507	2.13×10^{-4}	-0.8218	1.60×10^{-5}
SSB	-0.7021	8.06×10^{-4}	-0.8662	1.64×10^{-6}
SSE	0.7705	1.13×10^{-4}	0.8592	2.46×10^{-6}
XB	0.7617	1.51×10^{-4}	0.8547	3.16×10^{-6}

Tabla 3.11: Correlación entre métricas internas y costo total (heurística NearestNeighborWithRLC).

Métrica	Spearman	p-valor	Pearson	p-valor
BH	0.5426	1.64×10^{-2}	0.6879	1.13×10^{-3}
CH	-0.4583	4.85×10^{-2}	-0.6088	5.67×10^{-3}
DB	0.5274	2.03×10^{-2}	0.6908	1.06×10^{-3}
DI	-0.6959	9.36×10^{-4}	-0.8299	1.11×10^{-5}
RT	0.4241	7.04×10^{-2}	0.6254	4.19×10^{-3}
CS	-0.4943	3.15×10^{-2}	-0.6249	4.23×10^{-3}
SSB	-0.3975	9.19×10^{-2}	-0.6177	4.83×10^{-3}
SSE	0.4642	4.52×10^{-2}	0.6236	4.33×10^{-3}
XB	0.4133	7.86×10^{-2}	0.6200	4.63×10^{-3}

Tabla 3.12: Correlación entre métricas internas y costo total (heurística NearestNeighborWithRLC con RLC=1).

Métrica	Spearman	p-valor	Pearson	p-valor
BH	0.2828	2.41×10^{-1}	0.2447	3.13×10^{-1}
CH	-0.1353	5.81×10^{-1}	-0.1692	4.89×10^{-1}
DB	0.2555	2.91×10^{-1}	0.2648	2.73×10^{-1}
DI	-0.5610	1.25×10^{-2}	-0.6209	4.56×10^{-3}
RT	0.1036	6.73×10^{-1}	0.1863	4.45×10^{-1}
CS	-0.1827	4.54×10^{-1}	-0.1870	4.43×10^{-1}
SSB	-0.0061	9.80×10^{-1}	-0.1465	5.50×10^{-1}
SSE	0.1484	5.44×10^{-1}	0.1885	4.40×10^{-1}
XB	0.0957	6.97×10^{-1}	0.1805	4.60×10^{-1}

Los resultados revelan que todas las métricas internas presentan una correlación estadísticamente significativa con el costo total de las rutas con (valor $p < 0.05$) en la mayoría de las heurísticas de construcción evaluadas. La única excepción es la heurística NearestNeighborWithRLC con RLC=1, donde solo el Índice de Dunn (DI) mantiene significación estadística. Este resultado contrasta con la conclusión del estudio previo, donde se afirmaba que no existía correlación.

Interpretación de las correlaciones

Las correlaciones observadas siguen la dirección esperada según el criterio de cada métrica:

- Métricas de cohesión (SSE, BH): presentan correlación positiva con el costo. Es decir, valores más altos de dispersión intra-clúster se asocian con rutas más costosas. El Índice de Ball-Hall (BH) destaca con la correlación más fuerte (Pearson hasta 0.952 en SaveSequential y CMT), lo que lo posiciona como el mejor predictor del costo dentro de este conjunto.

- Métricas de separación (SSB): presentan correlación negativa. Una mayor separación entre clústeres se asocia con un menor costo de rutas.
- Métricas combinadas (CH, DI, CS): muestran correlación negativa con el costo. Los índices de Dunn (DI) y Calinski-Harabasz (CH) destacan por su capacidad predictiva (DI: Pearson hasta -0.847 en SaveParallel; CH: Pearson hasta -0.857 en MoleJameson).
- Índices DB, RT y XB: presentan correlación positiva. Valores más altos se asocian con costos más elevados.

Patrones por heurística

Al comparar las heurísticas de construcción, se observan patrones consistentes:

- SaveSequential, CMT y MoleJameson presentan las correlaciones más fuertes, con métricas como BH y DB alcanzando coeficientes superiores a 0.9.
- Sweep y SaveParallel muestran correlaciones sólidas y significativas en todas las métricas.
- NearestNeighborWithRLC es la heurística con correlaciones más débiles, y algunas de ellas no resultan significativas (RT, SSB, XB con Spearman $p > 0.05$).
- NearestNeighborWithRLC=1 presenta el caso más extremo: solo el Índice de Dunn mantiene correlación significativa. Esto sugiere que con un tamaño de lista de candidatos restringidos muy pequeño, las asignaciones pierden estructura y las métricas internas dejan de ser predictivas del costo.

En todos los casos, BH (Ball-Hall), DI (Dunn), CH (Calinski-Harabasz), DB (Davies-Bouldin) y SSE destacan como las métricas con mayor poder predictivo del costo total. Esto las posiciona como candidatas ideales para ser utilizadas como criterios de selección de algoritmos de asignación cuando no se dispone del costo de las rutas.

3.5 Conclusiones parciales

A partir del estudio experimental realizado en este capítulo, se pueden extraer las siguientes conclusiones:

- Las pruebas funcionales verificaron el correcto funcionamiento de los métodos esenciales de HDBSCAN, incluyendo el cálculo de distancias, la obtención de vecinos y la búsqueda del depósito más cercano, garantizando su integración fiable en BHAVRP.
- Las pruebas unitarias de caja negra confirmaron que la adaptación de HDBSCAN respeta las restricciones de capacidad y maneja adecuadamente escenarios extremos como datos erróneos, depósitos vacíos y clústeres no asignables.
- La comparación estadística mediante la prueba de Friedman y los análisis *post-hoc* reveló que CURE, UPGMC, KMEANS y Sweep conforman el grupo de mejor desempeño en la calidad de las rutas. HDBSCAN se posiciona en un nivel intermedio, superando significativamente a DBSCAN y a las heurísticas clásicas, y sin diferencias significativas con varios de los mejores algoritmos, lo que demuestra su rendimiento y su aporte como alternativa basada en densidad.
- El análisis de correlación demostró que las métricas internas de validación presentan correlación estadísticamente significativa con el costo total de las rutas en la mayoría de las heurísticas de construcción. Los índices de Ball-Hall y Dunn destacan como los mejores predictores, lo que confirma la utilidad práctica de las métricas internas para seleccionar algoritmos de asignación sin necesidad de construir las rutas.

Referencias Bibliográficas

[1] M. E. Coloma Salazar, J. Arzola Ruiz y C. E. Marrero Fornaris, "La Gestión de La Distribución y La Planificación de Rutas En El Sector Empresarial Cubano," *Revista Universidad y Sociedad*, vol. 14, págs. 357-368, 2022, ISSN: 2218-3620.

[2] G. B. Dantzig y J. H. Ramser, *The Truck Dispatching Problem. Management Science*, 6.ª ed. 1959, 80-91.

[3] G. Clarke y J. W. Wright, "Scheduling of Vehicles from a Central Depot to a Number of Delivery Points," *Operations Research*, vol. 12, n.º 4, págs. 568-581, 1964. DOI: 10.1287/opre.12.4.568

[4] S. Arora y B. Barak, *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.

[5] K. Braekers, K. Ramaekers e I. Van Nieuwenhuyse, "The vehicle routing problem: State of the art classification and review," *Computers & Industrial Engineering*, vol. 99, págs. 300-313, sep. de 2016, ISSN: 03608352. DOI: 10.1016/j.cie.2015.12.007 visitado 21 de ene. de 2026. dirección: https://linkinghub.elsevier.com/retrieve/pii/S0360835215004775

[6] D. I. G. Mateos, "DESARROLLO DE UN MÉTODO HÍBRIDO PARA LA RESOLUCIÓN DEL MDVRP," *Revista de la Escuela Jacobea de Posgrado*, 2013.

[7] J. A. G. Bermudez, "Aplicación de técnicas de clusterización como inicializadores y la metaheurística ILS, al Problema de Ruteo Vehículos con Múltiples Depósitos (MDVRP)," Proyecto de grado presentado como requisito parcial para optar al título de Magister en Ingeniería Eléctrica, UNIVERSIDAD TECNOLÓGICA DE PEREIRA.

[8] S. Barreto, C. Ferreira, J. Paixão y B. S. Santos, "Using Clustering Analysis in a Capacitated Location-Routing Problem," *European Journal of Operational Research*,

vol. 179, n.º 3, págs. 968-977, 2007, ISSN: 0377-2217. DOI: 10.1016/j.ejor.2005.06.074 dirección: https://www.sciencedirect.com/science/article/pii/S0377221706000786

[9] Y. Z. Mehrjerdi y A. Nadizadeh, "Using Greedy Clustering Method to Solve Capacitated Location-Routing Problem with Fuzzy Demands," *European Journal of Operational Research*, vol. 229, n.º 1, págs. 75-84, 2013, ISSN: 0377-2217. DOI: 10.1016/j.ejor.2013.02.013 dirección: https://www.sciencedirect.com/science/article/pii/S0377221713001318

[10] A. K. Jain y R. C. Dubes, *Algorithms for Clustering Data* (Prentice Hall Advanced Reference Series). Englewood Cliffs, NJ: Prentice-Hall, 1988, 320 págs., ISBN: 978-0-13-022278-7.

[11] K. Khan, S. U. Rehman, K. Aziz, S. Fong y S. Sarasvady, "DBSCAN: Past, Present and Future," en *The Fifth International Conference on the Applications of Digital Information and Web Technologies (ICADIWT 2014)*, IEEE, 2014, págs. 232-238.

[12] J. MacQueen, "Some Methods for Classification and Analysis of Multivariate Observations," en *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, University of California Press, 1967, págs. 281-297.

[13] P. Paduloh y T. Djatna, "A Robust Optimizing Reverse Logistics Model for Beef Products Using Multi Depot Vehicle Routing Problem," *ComTech: Computer, Mathematics and Engineering Applications*, vol. 14, n.º 1, págs. 45-54, 2023.

[14] J. Luo y M.-R. Chen, "Multi-Phase Modified Shuffled Frog Leaping Algorithm with Extremal Optimization for the MDVRP and the MDVRPTW," *Computers & Industrial Engineering*, vol. 72, n.º 1, págs. 84-97, 2014. DOI: 10.1016/j.cie.2014.03.004

[15] S. Geetha, G. Poonthalir y P. T. Vanathi, "Nested Particle Swarm Optimisation for Multi-Depot Vehicle Routing Problem," *International Journal of Operational Research*, vol. 16, n.º 3, págs. 329-348, 2013. DOI: 10.1504/IJOR.2013.052336

[16] "GitHub - Vrpsolutions/BHAVRP: BHAVRP: Componente de Software Para La Asignación de Clientes a Depósitos En Problemas de Rutas de Vehículos. Implementa Heurísticas Clásicas y Algoritmos de Agrupamiento Para Resolver La Fase de Asignación de Un MDVRP, Diseñada Para Aplicaciones de Logística y Planificación de Rutas.," visitado 21 de ene. de 2026. dirección: https://github.com/vrpsolutions/BHAVRP

[17] Eric Ramos Aragón, "Incorporación de Un Módulo de Métricas de Evaluación Para Algoritmos de Asignación En BHAVRP," Trabajo para optar por el título de Ingeniero Informático, Universidad Tecnológica de la Habana "José Antonio Echeverría", La Habana, Cuba, 2025.

[18] S. N. Kumar, "A Survey on the Vehicle Routing Problem and Its Variants," *Intelligent information management*, 2012.

[19] G. Berbeglia, J.-F. Cordeau y G. Laporte, "Dynamic Pickup and Delivery Problems," *European Journal of Operational Research*, vol. 202, n.º 1, págs. 8-15, 2007. DOI: 10.1016/j.ejor.2009.04.024

[20] R. Baldacci, M. Battarra y D. Vigo, "Routing a Heterogeneous Fleet of Vehicles," en *The Vehicle Routing Problem: Latest Advances and New Challenges*, B. Golden, S. Raghavan y E. Wasil, eds., Springer, 2008, págs. 3-27. DOI: 10.1007/978-0-387-77778-8_1

[21] D. Sariklis y S. Powell, "A Heuristic Method for the Open Vehicle Routing Problem," *Journal of the Operational Research Society*, vol. 51, n.º 5, págs. 564-573, 2000. DOI: 10.1057/palgrave.jors.2600924

[22] J.-F. Cordeau, G. Desaulniers, J. Desrosiers, M. M. Solomon y F. Soumis, "The VRP with Time Windows," en *The Vehicle Routing Problem*, P. Toth y D. Vigo, eds., SIAM, 2002, págs. 157-193. DOI: 10.1137/1.9780898718515.ch7

[23] J. R. Montoya-Torres, J. López Franco, S. Nieto Isaza, H. Felizzola Jiménez y N. Herazo-Padilla, "A Literature Review on the Vehicle Routing Problem with Multiple

Depots," *Computers & Industrial Engineering*, vol. 79, págs. 115-129, 2015. DOI: 10.1016/j.cie.2014.10.029

[24] M. Gendreau, G. Laporte y R. Séguin, "Stochastic Vehicle Routing," *European Journal of Operational Research*, vol. 88, n.º 1, págs. 3-12, 1996. DOI: 10.1016/0377-2217(95)00050-X

[25] P. M. Francis, K. R. Smilowitz y M. Tzur, "The Period Vehicle Routing Problem and Its Extensions," en *The Vehicle Routing Problem: Latest Advances and New Challenges*, B. Golden, S. Raghavan y E. Wasil, eds., Springer, 2008, págs. 73-102. DOI: 10.1007/978-0-387-77778-8_4

[26] A. Ibrahim, R. Abdulaziz y J. Ishaya, "CAPACITATED VEHICLE ROUTING PROBLEM," *International Journal of Research - GRANTHAALAYAH*, vol. 7, págs. 310-327, 12 de abr. de 2019. DOI: 10.5281/zenodo.2636820

[27] V. Rebillas Loredo, "The Multi-Depot VRP with Vehicle Interchanges," Tesis doct., Universitat Politècnica de Catalunya, 2018. DOI: 10.5821/dissertation-2117-127490 dirección: https://www.tdx.cat/handle/10803/664634

[28] J. B. Colin, "Diseño de un algoritmo metaheurístico para la solución del Problema de Ruteo de Vehículos MultiDepósito," PARA OBTENER EL GRADO DE MAESTRÍA EN OPTIMIZACIÓN Y CÓMPUTO APLICADO, UNIVERSIDAD AUTÓNOMA DEL ESTADO DE MORELOS, 2024.

[29] J.-F. Cordeau y M. Maischberger, "A Parallel Iterated Tabu Search Heuristic for Vehicle Routing Problems," *Computers & Operations Research*, vol. 39, n.º 9, págs. 2033-2050, 2012, ISSN: 0305-0548.

[30] T. Vidal, T. G. Crainic, M. Gendreau, N. Lahrichi y W. Rei, "A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems," *Operations Research*, vol. 60, n.º 3, págs. 611-624, 2012.

[31] A. Subramanian, E. Uchoa y L. S. Ochi, "A Hybrid Algorithm for a Class of Vehicle Routing Problems," *Computers & Operations Research*, vol. 40, n.º 10, págs. 2519-2531, 2013, ISSN: 0305-0548. dirección: http://www.sciencedirect.com/science/article/pii/S030505481300021X

[32] J. W. Escobar, R. Linfati, P. Toth y M. G. Baldoquin, "A Hybrid Granular Tabu Search Algorithm for the Multi-Depot Vehicle Routing Problem," *Journal of Heuristics*, vol. 20, n.º 5, págs. 483-509, 2014, ISSN: 1381-1231. DOI: 10.1007/s10732-014-9247-0

[33] J.-F. Cordeau, M. Gendreau y G. Laporte, "A Tabu Search Heuristic for Periodic and Multi-Depot Vehicle Routing Problems," *Networks*, vol. 30, n.º 2, págs. 105-119, 1997. DOI: 10.1002/(SICI)1097-0037(199709)30:2<105::AID-NET5>3.0.CO;2-G

[34] F. B. de Oliveira, "Cooperative coevolutionary models for the multi-depot vehicle routing problem," Tesis doct., Universidade Federal de Minas Gerais (UFMG), Belo Horizonte, MG, Brazil, 2015.

[35] F. Hamid, ed., *Optimization Essentials: Theory, Tools, and Applications* (International Series in Operations Research & Management Science volume 353). Singapore: Springer, 2024, 798 págs., ISBN: 978-981-99-5491-9 978-981-99-5490-2.

[36] M. Jünger et al., "50 Years of Integer Programming 1958-2008: From the Early Years to the State-of-the-Art," 2009, ISSN: 978-3-540-68274-5. DOI: 10.1007/978-3-540-68279-0 dirección: https://www.springer.com/gp/book/9783540682745

[37] D. Mogale, A. Ghadge y S. K. Jena, "Modelling and Optimising a Multi-Depot Vehicle Routing Problem for Freight Distribution in a Retail Logistics Network," *Computers & Industrial Engineering*, vol. 207, pág. 111 315, 1 de sep. de 2025, ISSN: 0360-8352. DOI: 10.1016/j.cie.2025.111315 dirección: https://www.sciencedirect.com/science/article/pii/S0360835225004619

[38] E. Erdem, T. Aydin y B. Erkayman, "AI-Infused Evolutionary Optimization for Multi-Depot Vehicle Routing," *A Case Study of Beverage Distribution*, vol. 37, n.º 27–28, 25 de dic. de

2025, ISSN: 1532-0626. DOI: 10.1002/cpe.70431 dirección: https://www.scopus.com/pages/publications/105021620759

[39] E.-G. Talbi, *Metaheuristics: From Design to Implementation*. Hoboken, New Jersey: John Wiley & Sons, Inc., 2009, ISBN: 978-0-470-27858-1.

[40] F. S. Hernández, "APLICACIÓN DEL MODELO VRP (VEHICLE ROUTING PROBLEM) PARA LA OPTIMIZACIÓN DE UNA RED DE DISTRIBUCIÓN," UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO, 2015.

[41] I.-M. Chao, B. L. Golden y E. A. Wasil, "A New Heuristic for the Multi-Depot Vehicle Routing Problem That Improves upon Best-Known Solutions," *American Journal of Mathematical and Management Sciences*, vol. 13, págs. 371-406, 1993. dirección: https://api.semanticscholar.org/CorpusID:62687165

[42] F. A. Tillman, "The Multiple Terminal Delivery Problem with Probabilistic Demands," *Transportation Science*, vol. 3, n.º 3, págs. 192-204, 1969. DOI: 10.1287/trsc.3.3.192

[43] B. E. Gillett y L. R. Miller, "A Heuristic Algorithm for the Vehicle-Dispatch Problem," *Operations Research*, vol. 22, n.º 2, págs. 340-349, 1974. DOI: 10.1287/opre.22.2.340

[44] Y. Shi, L. Lv, F. Hu y Q. Han, "A Heuristic Solution Method for Multi-Depot Vehicle Routing-Based Waste Collection Problems," *Applied Sciences*, vol. 10, n.º 7, 2020, ISSN: 2076-3417. DOI: 10.3390/app10072403 dirección: https://www.mdpi.com/2076-3417/10/7/2403

[45] C. Blum y A. Roli, "Metaheuristics in combinatorial optimization: Overview and conceptual comparison," *ACM Computing Surveys*, vol. 35, n.º 3, págs. 268-308, sep. de 2003, ISSN: 0360-0300, 1557-7341. DOI: 10.1145/937503.937505 visitado 19 de mar. de 2026. dirección: https://dl.acm.org/doi/10.1145/937503.937505

[46] J. H. Orallo, M. J. R. Quintana y C. F. Ramírez, *Introducción a La Minería de Datos*. Pearson Educación, 2004.

[47] G. Kesavaraj y S. Sukumaran, "A Study on Classification Techniques in Data Mining," en *2013 Fourth International Conference on Computing, Communications and Networking Technologies (ICCCNT)*, IEEE, 2013, págs. 1-7.

[48] Ş. Şeker, A. F. Güneri, G. Ermiş y İ. Yıldız, "Solving the Multi Depot Vehicle Routing Problem with Assignment Algorithm," en *13th International Research/Expert Conference "Trends in the Development of Machinery and Associated Technology", Hammamet*, 2009, págs. 225-228.

[49] K. Koutroumbas y S. Theodoridis, *Pattern Recognition*. Academic Press, 2008.

[50] C. Gao et al., "An Overview of Clustering Methods with Guidelines for Application in Mental Health Research," *Psychiatry Research*, vol. 327, pág. 115 265, 1 de mayo de 2023. DOI: 10.1016/j.psychres.2023.115265

[51] S. Salhi y M. Sari, "A Heuristic Algorithm for the Multi-Depot Vehicle Routing Problem," *Journal of the Operational Research Society*, vol. 49, n.º 4, págs. 424-431, 1998. DOI: 10.1057/palgrave.jors.2600526

[52] JSprit Contributors. "JSprit: A Java-based Library for Solving Vehicle Routing Problems," visitado 18 de mar. de 2026. dirección: https://jsprit.readthedocs.io/en/latest/

[53] L. Perron y V. Furnon, *OR-Tools*, ver. v9.7, Google. dirección: https://developers.google.com/optimization/

[54] M. Karkula, J. Duda e I. Skalna, "Comparison of capabilities of recent open source tools for solving capacitated vehicle routing problems with time windows," en *Carpathian Logistics Congress*, 2019.

[55] F. Pedregosa, *Scikit-Learn: Machine Learning in Python.* 12.ª ed. 2011, 2825-2830.

[56] Ananda de la Caridad Morales Morales, “Nueva Versión de La Biblioteca de Heurísticas de Construcción Para Problemas de Planificación de Rutas de Vehículos,” Trabajo para optar por el título de Ingeniero Informático, Universidad Tecnológica de la Habana “José Antonio Echeverría”, La Habana, Cuba, 2025.

[57] G. Yañez e I. Torres, “Desarrollo de Nuevos Modelos Para El Problema Del Transporte Obrero En CaSVRP.,” Universidad Tecnológica de la Habana “José Antonio Echeverría”, La Habana, Cuba, 2021.

[58] R. D. Hernández, “Nueva Versión de La Biblioteca de Clases BiCIAM Para Solucionar Problemas Multiobjetivo,” Trabajo de diploma, Universidad Tecnológica de La Habana “José Antonio Echeverría”, La Habana, Cuba, 2014.

[59] L. Lores, “Operadores de Mutación Basados En Heurísticas de Construcción Para Problemas de Planificación de Rutas de Vehículos,” Trabajo de diploma (Ingeniería Informática), Universidad Tecnológica de La Habana “José Antonio Echeverría”, La Habana, Cuba, 2017.

[60] M. Ester, H.-P. Kriegel, J. Sander, X. Xu et al., “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise,” en *Kdd*, vol. 96, 1996, págs. 226-231.

[61] M. Ankerst, M. M. Breunig, H.-P. Kriegel y J. Sander, “OPTICS: Ordering Points to Identify the Clustering Structure,” en *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*, ACM, 1999, págs. 49-60.

[62] R. J. Campello, D. Moulavi y J. Sander, “Density-Based Clustering Based on Hierarchical Density Estimates,” en *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, Springer, 2013, págs. 160-172.

[63] A. Hinneburg y D. A. Keim, “An Efficient Approach to Clustering in Large Multimedia Databases with Noise,” en *Proceedings of the 4th International Conference on Knowledge Discovery and Data Mining (KDD-98)*, vol. 98, 1998, págs. 58-65.

[64] A. Rodriguez y A. Laio, "Clustering by Fast Search and Find of Density Peaks," *Science*, vol. 344, n.º 6191, págs. 1492-1496, 2014.

[65] R. C. Prim, "Shortest Connection Networks and Some Generalizations," *Bell System Technical Journal*, vol. 36, n.º 6, págs. 1389-1401, 1957. DOI: 10.1002/j.1538-7305.1957.tb01515.x

[66] E. Raff, *JSAT: Java Statistical Analysis Tool*. dirección: https://github.com/EdwardRaff/JSAT

[67] M. L. Fredman y R. E. Tarjan, "Fibonacci Heaps and Their Uses in Improved Network Optimization Algorithms," *Journal of the ACM*, vol. 34, n.º 3, págs. 596-615, 1987. DOI: 10.1145/28869.28874

[68] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 3rd. Cambridge, MA: MIT Press, 2009, ISBN: 978-0-262-03384-8.

[69] B. A. Galler y M. J. Fischer, "An Improved Equivalence Algorithm," *Communications of the ACM*, vol. 7, n.º 5, págs. 301-303, 1964. DOI: 10.1145/364099.364331

[70] R. E. Tarjan y J. van Leeuwen, "Worst-Case Analysis of Set Union Algorithms," *Journal of the ACM*, vol. 31, n.º 2, págs. 245-281, 1984. DOI: 10.1145/62.2160

[71] K. Abduaziz, S. Makhammadjonov y J. S. Shin, "Minimal and Simplified Analysis of Hierarchical Density-Based Spatial Clustering," *IEEE access : practical innovations, open solutions*, vol. 13, págs. 145 191-145 199, 2025.

[72] E. R. Aragón, I. T. Pérez, A. R. Suárez y A. d. l. C. M. Morales, "BHAVRP: Biblioteca de Heurísticas de Asignación Para El Problema de Planificación de Rutas de Vehículos Con Múltiples Depósitos," *Revista Cubana de Ingeniería (RCI)*, págs. 1-13, 2025.

[73] M. Friedman, "The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance," *Journal of the American Statistical Association*, vol. 32, n.º 200, págs. 675-701, 1937.

[74] B. Bergmann y G. Hommel, "Improvements of General Multiple Test Procedures for Redundant Systems of Hypotheses," en *Multiple Hypothesenprüfung — Multiple Hypotheses Testing*, Berlin, Heidelberg: Springer-Verlag, 1988, págs. 100-115. DOI: 10.1007/978-3-642-52307-6_8 dirección: https://doi.org/10.1007/978-3-642-52307-6_8

[75] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian Journal of Statistics*, vol. 6, págs. 65-70, 1979. JSTOR: 4615733. dirección: https://www.jstor.org/stable/4615733

[76] J. P. Shaffer, "Modified Sequentially Rejective Multiple Test Procedures," *Journal of the American Statistical Association*, vol. 81, n.º 395, págs. 826-831, 1986. DOI: 10.1080/01621459.1986.10478341 dirección: https://doi.org/10.1080/01621459.1986.10478341